

Determinismo antes que selección: qué es aprendible en la orquestación de agentes LLM

Borrador 0.2 — 2026-08-30 Autor: Ariel Edgardo Levy **Estado:** borrador de trabajo. La teoría está verificada por máquina; las mediciones empíricas son de **78 tareas × 12 paradigmas × 3 réplicas** sobre un corpus con entidades reales (121,4M tokens). Toda afirmación empírica lleva su tamaño de muestra. Lo que aún **no** hay es una medición válida sobre datos held-out; está declarado donde corresponde. Destino: arXiv cs.LG (primario), cs.AI (cross-list).

Traducción de `paper-en.md`, **que es la versión canónica**. Si difieren, manda el inglés.

Resumen

Este trabajo empezó preguntando **qué paradigma de orquestación conviene por tarea** y termina con una respuesta distinta a la que buscaba. Sobre un registro completo —78 tareas × 12 paradigmas × 3 réplicas, sin juez LLM y con el corrector auditado— **el ruteo por calidad no tiene premio neto:** $-0,008$ contra el mejor paradigma fijo, y el mecanismo del fracaso es más informativo que el número. Lo que sí queda en pie, y es lo que el paper sostiene, son dos afirmaciones sobre **determinismo** y una sobre **qué es lo aprendible**.

Primero: la interacción existe, es enorme, y no es cobrable. Descomponiendo $u = \mu + \alpha(\text{tarea}) + \beta(\text{brazo}) + \gamma + \epsilon$, la interacción γ explica el **48%** de la varianza con señal/ruido $5,30$, y una señal la predice —`cardinalidad × término literal`, que captura dos tercios del techo y **sobrevive la corrección por selección** contra el máximo de nueve nulos por permutación—. Y aun así el premio neto es negativo, porque **$\text{var}(\gamma)$ grande $\neq$ premio de ruteo grande**: γ es enorme porque los brazos *malos* son malos en lugares distintos. Restringida a los brazos que competirían, γ real cae a $-0,0046$ con señal/ruido $0,38$. Reportar varianza de interacción como evidencia de que rutear conviene mide la estructura equivocada. El resultado se replica a nivel de familias declaradas desde el código: $+0,079$ contra un piso de ruido de $+0,065$, y ninguna señal predice qué familia gana.

Segundo: el determinismo es el recurso escaso, y es recuperable. `pass^k` —acertar en las tres réplicas— cae entre $0,078$ y $0,196$ por brazo, con **17% a 34% de las celdas inestables** a temperatura cero, semilla fija y la misma huella. Ningún brazo alcanza reproducibilidad por decisión mayor a $0,983$. Pero se recupera **sacando decisiones de control del modelo y devolviéndolas al código**: cuatro correcciones estructurales —ninguna de fraseo— llevaron un brazo de $0,33$ a $0,89$ en la celda de cadenas acopladas y hicieron coincidir sus réplicas. La intervención decisiva es de una sola línea de diseño: con la misma huella y los mismos resultados de búsqueda, dejar que el modelo eligiera el ancla de una caminata daba $1,000 / 0,000 / 0,000$; que lo elija el código da $1,000$ en las tres. **La conjetura ingenua de que el no-determinismo se compone con el número de decisiones es falsa** ($r = -0,24$, $n = 8$): importa cuál decisión, no cuántas.

Tercero: lo aprendible es la capacidad, no la identidad del paradigma. El intento previo de mapear ontología de la pregunta → nombre de paradigma se refutó ($-0,087$). Se propone el eslabón que faltaba —ontología → **capacidades exigidas** → brazos que las tienen— con diez capacidades declaradas desde el código, cada una con su evidencia. La prueba correcta es **dejar un brazo afuera**, no una tarea: un modelo con identidad de paradigma no tiene parámetro para un brazo que no vio; uno con capacidades sí. Medido, las capacidades predicen un brazo no visto mejor que la dificultad de la tarea sola (MAE $0,233$ contra $0,257$, ganando 6 de 8 pliegues) y mejor que la identidad del brazo aunque ésta vea al brazo dejado afuera ($0,339$). Contra el nulo de capacidades barajadas da $p = 0,065$: **sugestivo y no establecido**, y ocho brazos son ocho puntos.

Como soporte quedan tres resultados anteriores que el registro completo confirma: la **factibilidad es aritmética** y poda topologías antes de gastar un token; el **Teorema del Valor de Selección** particiona el problema por verificabilidad y no por tipo de tarea; y la **superficie de herramientas** gobierna la varianza que suele atribuirse a la topología.

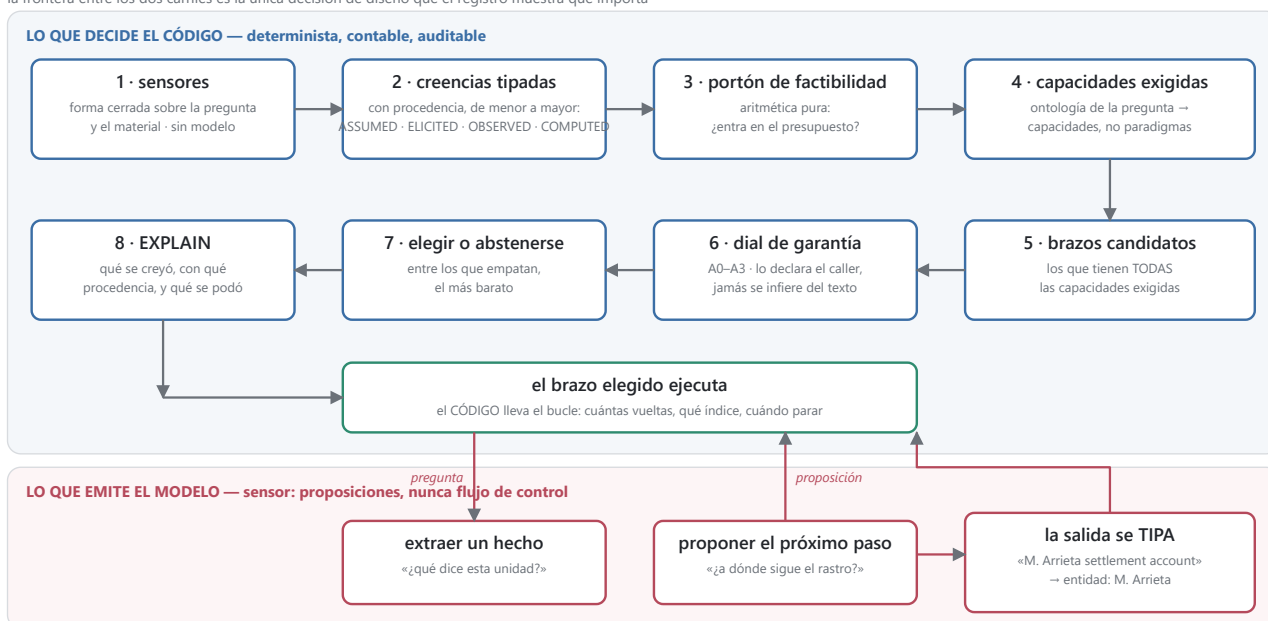
Contribuimos la teoría, una capa de medición cuyos ejes se contrastan explícitamente contra la literatura de benchmarks de agentes 2024-2026, un generador de corpus con ground truth exacto y entidades con variantes de superficie, un catálogo de capacidades declarado desde el código, y el análisis de fallas de doce topologías. **No** contribuimos todavía un selector validado sobre datos held-out.

Palabras clave: agentes LLM, determinismo, orquestación, patrones DAG, predicción selectiva, aprendizaje plástico, capacidades, evaluación de agentes

Cómo se decide un request

Cómo se decide un request

la frontera entre los dos carriles es la única decisión de diseño que el registro muestra que importa



Una decisión que cruza al carril de abajo se lleva el determinismo con ella: medido, el ancla de una caminata elegida por el modelo dio $u=1,000 / 0,000 / 0,000$ con la misma huella.

El método determinista: qué decide el código y qué emite el modelo

La figura es el argumento del paper en una imagen. El carril de arriba es determinista y auditable: sensores de forma cerrada, creencias tipadas con procedencia, un portón aritmético, las capacidades que la pregunta exige, y recién ahí una elección entre los brazos capaces —o una abstención—. El carril de abajo es el modelo, y **sólo emite proposiciones**: qué dice una unidad, hacia dónde sigue un rastro. Nunca decide cuántas vueltas dar, qué índice usar ni cuándo parar.

Toda decisión que cruza al carril de abajo se lleva el determinismo con ella, y eso está medido, no argumentado. La guarda que cierra el ciclo —tipar la salida del sensor antes de usarla— es del 2026-08-30 y salió de un caso concreto: el modelo emitía 'M. Arrieta settlement account' y esa cola arrastraba la consulta a clasificarse como prosa, mandándola al índice equivocado. **La regla era correcta y la entrada estaba sucia.**

lo que decide el código	lo que emite el modelo
cuántas vueltas dar (del largo declarado en la pregunta)	qué dice esta unidad
qué índice usar (léxico para entidad nombrada, híbrido para prosa)	hacia dónde sigue el rastro
qué unidad es el ancla, y cuál el término de la cadena	qué hecho aporta lo leído
si hay evidencia suficiente para responder, o si hay que abstenerse	la redacción de la respuesta

1. Introducción

1.1 Un premio medido que nadie cobra

El *paradigma* de un agente LLM — la estructura de control que envuelve al modelo: una llamada única, un bucle de razonamiento, una descomposición, un grafo de verificar-replanificar — normalmente se elige una vez en tiempo de diseño y se congela en el código. Trabajo reciente mide lo que eso cuesta. Sobre seis paradigmas, cuatro modelos frontera y diez benchmarks (~18.000 corridas), la selección oráculo por tarea supera al mejor paradigma fijo por **17,1pp** en promedio, con oscilaciones individuales de hasta +44pp y -15pp según el emparejamiento [Select-then-Solve, arXiv:2604.06753].

El mismo trabajo muestra que el premio no se está cobrando. Un ruteador entrenado sobre embeddings recupera aproximadamente un cuarto de la brecha. El auto-ruteo zero-shot — pedirle al modelo que elija su propio paradigma — recupera valor *negativo*: dos modelos caen por debajo de sus propias líneas base de paradigma único, uno hasta 27,5%.

Entonces: el premio es grande, el mejor intento publicado captura una minoría, y el intento ingenuo es peor que no intentar. Ese patrón invita a concluir que hacen falta mejores selectores. Nosotros sostenemos que invita a otra conclusión.

1.2 Tres afirmaciones que preceden al problema de aprendizaje

La factibilidad es aritmética, y es gratis. Antes de preguntar cuál topología es *mejor*, se puede preguntar cuál puede *correr*. Esa pregunta se responde con cantidades que la tarea ya declara. Una política aprendida que gasta episodios descubriendo que map-reduce pierde en tareas de 500 unidades está aprendiendo aritmética por el camino difícil; el tope era computable antes del primer token.

La selección paga sólo bajo una condición precisa. Un ruteador que debe elegir siempre no tiene ningún grado de libertad sobre su tasa de falsos positivos y por lo tanto paga cada error de ruteo. Formalizamos cuándo la selección le gana a un fallback fijo y mostramos que el punto de operación óptimo generalmente implica **abstenerse en la mayoría de las solicitudes** — lo que no es un ruteador más débil sino un objetivo distinto.

Mucho de lo que se le atribuye a la topología es atribuible a las herramientas. En nuestras mediciones los paradigmas que usan herramientas de recuperación abarcan un rango de costo de cincuenta veces mientras los que no lo hacen abarcan 1,2×. Un benchmark que no reporta la calidad de su superficie de herramientas no está comparando topologías; está comparando un retriever envuelto de siete maneras.

1.3 Qué establece y qué no establece este borrador

Establecido: la teoría (§5), verificada por máquina contra distribuciones sintéticas de respuesta conocida; una capa de factibilidad (§4) validada sobre cuatro escalas de corpus; un generador de corpus cuyo ground truth se re-deriva de forma independiente a partir de los documentos (§6).

Preliminar: cada número empírico de §7 y §8 es $n=1$ por celda, sobre un corpus, con un modelo. La varianza entre réplicas sobre la topología más elaborada se midió en hasta $3,93\times$ en costo, así que las magnitudes son indicativas. Los *mecanismos* son más robustos que las magnitudes, porque descansan en trazas de uso de herramientas y no en tamaños de efecto.

No establecido: un selector validado, ningún resultado sobre benchmarks públicos, y ninguna afirmación sobre otro régimen que no sea extracción de respuesta exacta sobre colecciones de documentos.

Estructuralmente no puesto a prueba, que es más fuerte que «no establecido» y salió de medir. §5.2 parte el problema según v , la disponibilidad de un detector barato en runtime, y manda $v = 1$ a una cascada y $v = 0$ a un ruteador. **Todos los corpus de este registro caen del lado $v = 1$** . No por elección: corregir sin juez significa corregir por coincidencia exacta, la coincidencia exacta necesita una respuesta de referencia, y el mismo campo se leía como el detector de runtime — así que 25 de 26 tareas por corpus declaraban $v = 1$. La regla de cascada dispara con prioridad más alta que la de selección, así que sobre dos corpus held-out independientes **la regla de selección no disparó nunca**, y los dos veredictos de ruteo son mediciones de la cascada.

La forma general es una advertencia sobre toda una clase de experimento, no sobre éste:

Un benchmark que establece corrección por coincidencia exacta contra una referencia **tiene un detector barato en cada tarea por construcción**, y por lo tanto no puede ejercitar la rama $v = 0$ de su propia partición. Ser corregible implica ser verificable.

Si la selección paga donde verificar es genuinamente imposible queda, en este registro, **abierto** — y necesita un corpus cuya disponibilidad de detector se declare por tarea en vez de heredarse de la clave de respuestas.

2. Trabajo relacionado

2.1 Búsqueda automática de workflows

AFlow reformula la optimización de workflows como búsqueda sobre workflows representados como código, con MCTS sobre operadores [arXiv:2410.10762]; ADAS y GPTSwarm buscan en espacios afines. Estos producen **un** workflow por benchmark, offline. Nuestro interés es la selección por solicitud dentro de un conjunto fijo y — antes de eso — qué miembros del conjunto pueden correr siquiera.

2.2 Selección de paradigma en tiempo de inferencia

Select-then-Solve entrena un ruteador liviano sobre embeddings para elegir un paradigma por tarea [arXiv:2604.06753]. FlowBank construye un portafolio offline y selecciona por consulta [arXiv:2606.11290]. TRACE-Router rutea a granularidad de traza de tarea y no de solicitud [arXiv:2607.22465]. Uno-Orchestra aprende una política conjunta de descomposición y despacho [arXiv:2605.05007].

Todos operan a **cobertura uno**: toda tarea recibe un paradigma. §5.1 sostiene que ésa es la restricción vinculante, más que la capacidad del modelo, y ninguno reporta una curva de riesgo-cobertura. Dos detalles de Select-then-Solve, verificados contra el paper completo y no contra su abstract (2026-08-26): su oráculo es un *máximo empírico sin corregir* por tarea, y sus propias limitaciones anotan que la muestra es fija, sin re-muestreo entre seeds — exactamente el sesgo hacia arriba que nuestro protocolo de piso de ruido (§6.1) existe para descontar. Y la maquinaria de deferral existe al lado, sin haber cruzado: ReDAct difiere decisiones individuales a un modelo más grande sobre un umbral calibrado de incertidumbre [arXiv:2604.07036]; el ruteo por descomposición de incertidumbre unifica abstención y ruteo con garantías distribution-free, para clasificadores [arXiv:2605.07805]; y el meta-ruteo composicional entrena un ruteador interpretable sobre features textuales y nombra un confidence gate con fallback a ruteo estático como trabajo futuro sin evaluar [arXiv:2608.00106]. Nadie aplica abstención a la selección de paradigmas (buscado 2026-08-26); la ventana se está cerrando a la vista.

2.3 Modelos de costo para workflows

GLOW predice performance de workflows agénticos a partir de features de grafo y de lenguaje [arXiv:2512.15751]; Cost-Aware Optimization for Agentic Query Execution hace explícita la analogía con la optimización clásica de consultas [arXiv:2606.03152]. Tomamos la analogía como establecida y no la reclamamos. Nuestro aporte está aguas arriba: un filtro duro de factibilidad que un modelo de costo no reemplaza, porque **un plan infactible no tiene costo**.

2.4 Gobernanza simbólica sobre inferencia probabilística

El Structured Cognitive Loop introduce Soft Symbolic Control, un mecanismo de gobernanza sobre inferencia probabilística [arXiv:2511.17673]. Es el trabajo más cercano a §6.2, y leído completo (2026-08-26) su gobernanza se parte en dos: *Regulation*, un metaprompt persistente en lenguaje natural cuya efectividad el propio paper anota que depende de qué tan bien el LLM interprete instrucciones, y *Control*,

un runtime determinista que aplica reglas duras fijas al historial de ejecución del turno — llamadas duplicadas, conteo de errores, profundidad de ciclos — no al contenido proposicional. Control sí carga una clasificación de riesgo por acción (cachear una lectura segura, bloquear una llamada crítica repetida, congelar para autorización humana), pero es una clase estática sobre acciones individuales dentro de un único loop fijo. Opera como **un modo global único**: no gradúa la garantía por solicitud, no deriva el nivel requerido de creencias sobre la solicitud, no restringe el espacio de planes admisibles por nivel, y no calibra la confianza declarada por el modelo. Esas cuatro cosas son lo que agregamos — sobre una base de creencias sobre la que decide la capa simbólica, no un metaprompt que se le pide al modelo obedecer.

Cerca, en la capa de creencias misma: Nous acota la confiabilidad de una creencia por la procedencia del canal [arXiv:2606.22030], MemIR tipa la memoria por procedencia para impedir el colapso de fuentes [arXiv:2605.25869], Eywa promueve hechos sólo cuando pasan validadores contra evidencia inmutable [arXiv:2605.30771], y HEP hace auditable la evolución de hipótesis — aunque toda evidencia validada mueve la creencia por igual, sin jerarquía de procedencia que gatee la promoción [arXiv:2607.09195]. El problema del jardín de senderos que se bifurcan — agentes que proponen y puntúan hipótesis sobre los mismos datos — está diagnosticado empíricamente en [arXiv:2607.01507] sin un mecanismo; la partición proponer/puntuar de §6.3 es uno.

MINERVA/HADD es el antecedente más cercano de la capa de creencias misma, y leído completo (2026-08-26) aporta **mecanismo, no vocabulario** [Zenodo 10.5281/zenodo.20003407]. Los invariantes HADD ya contienen: el LLM confinado a sensor tipado que nunca toma decisiones de flujo de control; una capa de cognición determinista que es función pura de la base de creencias — "mismo estado → misma acción", que es la forma de garantía que §6.2 adopta —; una compuerta epistémica de admisión de creencias en la frontera de percepción (el EVR gate, especificado en un paper companion [Zenodo 10.5281/zenodo.19791686]); un historial de creencias append-only que un auditor puede reproducir; credencia numérica por creencia con calibración adaptativa (EMRE); y el encuadre Kautz Tipo-2. Su alcance es general — la generación de metas, la selección de planes y el control de ejecución son todos deterministas sobre creencias de estado concreto — dentro de una librería de planes pre-verificada y una topología fija.

Lo que HADD no tiene es lo que §6.2 agrega: la procedencia como **jerarquía tipada con semántica de admisibilidad** y no como campo de trazabilidad; la regla de que las afirmaciones elicítadas son inadmisibles para acciones irreversibles (el dispositivo más cercano de HADD es la confirmación humana obligatoria — un mecanismo de consentimiento, no epistémico); la garantía **graduada por solicitud** — en HADD la garantía es uniforme, y su único dial adaptativo es un umbral de escalación por tenant; el espacio de **patrones acotado por nivel** — HADD acota su librería de planes globalmente; y la calibración **medida** (EMRE aprende confianza pero nunca la mide — sin ECE, sin diagramas de confiabilidad). La selección de paradigma, la abstención y el deferral quedan enteramente fuera de su alcance.

Contratos de delegación e identidad atestiguada. El vecino más cercano del lado del ruteo es la *paradoja de la procedencia* en ruteo multi-agente [arXiv:2603.18043], y está lo bastante cerca como para que la superposición se diga en vez de dejársela a un revisor. Su resultado es que rutear sobre calidad **auto-reportada** selecciona a los peores delegados y rinde peor que al azar (0,55 contra 0,68), y su remedio es gobernanza determinista: contratos de delegación que acotan la autoridad con objetivos, presupuestos y políticas de falla explícitos, más un modelo de identidad **reclamada contra atestiguada** para que el ruteo consuma métricas verificadas y no reclamamos. Es empírico, con delegados simulados y modelos reales, y el brazo atestiguado llega a ruteo casi óptimo.

De ahí salen dos cosas, y una de ellas achica lo que podemos afirmar.

Corrobora la disciplina del sensor desde una dirección adversarial. Nuestra razón para rechazar la auto-evaluación del modelo como entrada admisible es epistémica — una afirmación sobre su propia suficiencia no tiene procedencia por encima de **ELICITED**. La de ellos es adversarial: un delegado tiene incentivo a inflar. Las dos llegan a la misma prohibición, y **reclamada / atestiguada** se parece a **ELICITED / OBSERVED** restringido a una sola proposición.

Ocupa «procedencia + ruteo + contratos» como frase, así que la conjunción hay que enunciarla por lo que excluye. Su procedencia es una propiedad del **reclamo de calidad de un delegado**; la nuestra es un orden sobre **tipos de evidencia**, y ese orden es lo que una regla lee. No reportan retículo, ni piso sobre acciones irreversibles, ni curva riesgo-cobertura — miden exactitud de ruteo. Lo que queda nuestro es la conjunción de: un **retículo de procedencia sobre evidencia**, un **piso que gatea acciones irreversibles** con él, la abstención tasada como **curva riesgo-cobertura medida**, y el mismo cálculo aplicado a factibilidad, control y contenido. Cualquier término suelto de eso tiene antecedentes.

2.5 Predicción selectiva y aprender a diferir

Nuestra teoría es una aplicación de un marco establecido. La regla de Chow da el rechazo óptimo; Mozannar y Sontag dan un surrogate consistente para diferir a un experto [PMLR v119]; Verma y Nalisnick agregan deferral one-vs-all calibrado [arXiv:2202.03673]; Mohri y colegas dan formulaciones multi-experto principiadas. Contribuimos la aplicación a la selección de paradigmas y los corolarios específicos de §5.1, no el marco.

2.6 Agentes que aprenden sin actualizar pesos

Los enfoques de memoria experiencial — Expel, aprendizaje reflexivo experiencial, MemSkill, R²-Mem — acumulan insights, reglas o entradas de memoria. Sleep-time compute corre inferencia en tiempo ocioso y reporta ~1/5 de los tokens en inferencia [Letta]; SCM y trabajo afín agregan consolidación de inspiración biológica [arXiv:2604.20943, arXiv:2605.26099]. Todos consolidan **contenido**. §6.3 consolida la **política de control**, y la afirmación sobrevive una búsqueda fechada (2026-08-26) sólo como conjunción, así que la enunciamos como tal: la política se aprende de los episodios propios del agente, se consolida offline, en un artefacto determinista versionado que se ejecuta fuera del LLM, sobre features estructurales de la tarea. Cada conjunto tiene un vecino fuerte. Trace2Policy destila control desde trazas de *expertos* a bases de reglas que se reinyectan *como texto de prompt* [arXiv:2606.10457]; la compilación declarativa de políticas le da a las políticas de orquestación exactamente la forma de artefacto que queremos — determinista, auditable, versionada — *pero escrita por humanos*, no aprendida [arXiv:2603.27299]; el meta-ruteo composicional aprende un ruteador interpretable offline, sobre features textuales y no estructurales [arXiv:2608.00106]. Ninguno aprende su propia política de control hacia un artefacto así.

2.7 Evaluación y jueces

Deliberadamente **no usamos juez LLM**. Un estudio a gran escala sobre 21 modelos y 541.000 juicios reporta confiabilidad sin validez, y que el acuerdo crudo sobreestima la capacidad discriminativa [arXiv:2606.19544]. Las métricas estilo RAGAS exhiben sesgo de posición, de verbosidad y de auto-preferencia, y la mitigación recomendada es corridas repetidas con inspección de dispersión. Como nuestros tamaños de efecto son de un dígito de puntos porcentuales y la varianza del juez es del mismo

orden — y como el sesgo de verbosidad favorecería sistemáticamente a los paradigmas caros cuyo valor está justamente en cuestión — un juez introduciría **un sesgo alineado con la hipótesis**. §6.1 explica la alternativa.

Sobre atribución de fallos, MemFail aísla los fallos de sistemas de memoria en modos de resumen, almacenamiento, recuperación y razonamiento, y puede atribuir un error a uno de ellos sólo porque las operaciones intermedias quedan registradas [arXiv:2605.26667] — convergente con el requisito de §8.4, aunque su atribución corre sobre un juez LLM donde la nuestra corre sobre trazas deterministas de herramientas. Su titular es también el nuestro en miniatura: escalar las memorias recuperadas o la fuerza del modelo rinde poco y a veces degrada, dependiendo de la tarea.

2.8 Posicionamiento

	cobertura	decisión determinista	auditable	filtro de factibilidad	abstención
AFlow / ADAS / GPTSwarm	offline, un workflow	no	no	no	n/c
Select-then-Solve	1,0	no	no	no	no
FlowBank	1,0	no	no	no	no
TRACE-Router	1,0	no	no	no	no
SCL	modo global	sí	sí	no	no
Este trabajo	selectiva	sí	sí	sí	sí

3. Preliminares

Una **tarea** t aporta una pregunta, un conjunto de *unidades* (documentos), un presupuesto declarado de tokens, y flags de irreversibilidad y escritura de estado compartido. Un **paradigma** $p \in P$ es una estructura de control que puede llamar herramientas y debe emitir una respuesta. La **calidad** $q(t, p) \in [0, 1]$ es F1 de conjuntos contra un oráculo exacto. El **costo** $c(t, p)$ es el total de tokens sobre cada llamada que el paradigma hace.

El **mejor paradigma fijo** es $p^* = \operatorname{argmax}_p E_t[u(t, p)]$. El **oráculo** es $E_t[\max_p u(t, p)]$, y la **brecha del oráculo** es su diferencia. La utilidad es calidad neta de una preferencia de costo:

$$u(t, p) = q(t, p) - \lambda \cdot (c(t, p) / \min_{p'} c(t, p') - 1)$$

Normalizar por el paradigma más barato en esa tarea hace a λ interpretable — la calidad que uno está dispuesto a cambiar por un múltiplo extra del costo mínimo — y $\lambda=0$ recupera calidad pura exactamente.

No se elige ningún λ : la calidad y el costo crudos se guardan sin modificar y el trade-off se aplica en tiempo de análisis, de modo que los resultados se reportan como función de λ y no bajo un supuesto sobre λ .

Los siete paradigmas son Direct, CoT, ReAct, Map-Reduce, Plan-Execute, Reflection, y un DAG con verificar-replanificar sobre un blackboard compartido. Cinco espejan la grilla de Select-then-Solve para poder cotejar números contra los suyos; Map-Reduce se agrega porque debería ganar en cardinalidad alta, y el DAG porque una comparación que omite la topología más elaborada disponible está sesgada a favor de las simples.

4. La factibilidad es aritmética

4.1 El chequeo

Que p pueda correr t depende de cantidades que t ya declara. Con n unidades, contenido C tokens, presupuesto B , y una asignación $A = 0,6 \cdot B$ que deja lugar para la conversación:

paradigma	restricción vinculante	infactible cuando
Direct, CoT	un prompt con todas las unidades	$C > A$
Map-Reduce	una llamada por unidad, más un reduce sobre todos los parciales	$n > 80$ o $n \cdot f > A$
DAG	sub-preguntas $\times$ iteraciones $\times$ replanificaciones	llamadas proyectadas > 200
ReAct, Reflection	tope propio de iteraciones; lee selectivamente	nunca

donde f es el tamaño proyectado de un hallazgo. Sin llamada al modelo, sin estadística, sin aprendizaje.

4.2 Separa dos modos de falla que se confunden de rutina

Aplicado a **168 tareas sobre cinco corpus** del mismo generador, cada tarea con su propio presupuesto declarado:

corpus	tokens/unidad	unidades/tarea	contenido máx	Direct, CoT	Map-Reduce
gold_v2	232	60	16k	39/39	39/39
gold_v3	2.147	60	152k	12/39	39/39
gold_wide	264	500	135k	20/32	18/32
gold_deep	7.161	60	483k	2/26	26/26
gold_xl	2.499	500	1.272k	8/32	18/32

Tareas factibles sobre el total. Los otros cuatro paradigmas son factibles en **168/168**.

Agregado sobre paradigmas, el espacio de planes admisibles se contrae monótonamente con la escala — **273/273 celdas en gold_v2, 186/224 en gold_wide, 134/182 en gold_deep, 162/224 en gold_xl**: de 100% admisible a 72%, íntegramente por aritmética y antes de gastar un token. Un selector que opere sin esta capa tendría que aprender que ese cuarto del espacio es inalcanzable, un episodio fallado a la vez.

El par decisivo es gold_wide contra gold_deep. gold_deep lleva 3,6× más contenido, y Map-Reduce pasa de 18/32 factibles a **26/26** mientras Direct y CoT se derrumban a 2/26. El tamaño total no predice la factibilidad de Map-Reduce; **la cardinalidad sí**. Con 483k tokens en 60 unidades corre, porque nunca las tiene juntas. Con 135k en 500 unidades no: 501 llamadas, y un reduce que concatena 500 hallazgos. Tratar su límite como un límite de contexto lleva a descartarlo exactamente donde funciona.

El mismo tope de acumulación aplica al blackboard del DAG, que renderiza cada hallazgo dentro de cada prompt de sub-agente.

Cuán fuerte muerde la restricción de leer-todo es fácil de subestimar. En gold_deep una tarea de **una sola unidad** es infactible para Direct, porque un documento son 8.075 tokens contra una asignación de 4.800. La restricción no es "el corpus es grande" sino "la pieza más chica que se puede direccionar ya no entra", y ninguna cantidad de lectura selectiva cambia eso para un paradigma cuyo único movimiento es leer.

Una segunda lectura que conviene decir en voz alta. Los cuatro paradigmas selectivos son factibles en las 168 tareas. Es una propiedad real — acotan sus propias iteraciones y leen a demanda — pero también acota lo que esta capa puede hacer. La factibilidad restringe sólo a los paradigmas que retienen material en contexto o se abren por unidad; **no ofrece ninguna protección contra un paradigma selectivo gastando a través de un corpus de 1,27M tokens.** Esa protección tiene que venir de un presupuesto, no de aritmética sobre el corpus, y §7.2 muestra por qué hace falta: los selectivos son justamente los que varían cincuenta veces en costo.

4.3 Por qué esto va delante del problema de aprendizaje

La factibilidad es determinista, gratis, y aguas arriba de todo lo demás. Poda el espacio de planes antes de cualquier selección, aprendida o no. **El corolario para producción es que saber el largo y declinar no es una degradación** — es la diferencia entre un sistema acotado y uno desbocado.

\1

Medido, y la distinción no es académica. Sobre cuatro celdas de gold_deep, Direct quedó podado en tres y corrió en una, donde sacó 1,000. Registradas como respuestas equivocadas, esas tres dejarían la media de Direct en 0,250 — el peor paradigma de la tabla. Registradas como infactibles, el enunciado es el correcto: *el mejor donde puede correr, no disponible donde no*. La misma capa que protege al estudio de una conclusión falsa es la que un sistema en producción necesita para declinar en lugar de fallar.

5. Teoría

Vecinos leídos el 2026-08-28, y lo que le sacan a la afirmación. Quedaban cuatro papers. **Tres ocupan, cada uno por su lado, mecanismos que este paper venía tratando como propios.**

trabajo	qué ocupa
EnvProbe — <i>Ask the World Before Acting: Budgeted Environment Probing for World-Model Calibration</i> (arXiv 2606.31422)	un operador de sondeo con presupuesto cuyo único propósito es reparar una tabla de creencias estructurada . Es nuestra sonda, mecanismo por mecanismo
Kintsugi — <i>Learning Policies by Repairing Executable Knowledge Bases</i> (arXiv 2605.09487)	ediciones a un artefacto ejecutable tipado, gateadas por un verificador , con las fallas diagnosticadas y localizadas en ediciones candidatas. Es nuestra consolidación con su guarda de promoción
ProvenanceGuard — <i>Safeguarding LLM Agents from Misalignment through Provenance Analysis</i> (arXiv 2607.01236)	la desalineación como si una llamada propuesta está sostenida por evidencia trazable en el contexto . Es nuestro piso de procedencia sobre las acciones

Y el área está lo bastante poblada como para tener **survey**: *From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents* (arXiv 2606.04990).

Así que la posición honesta es más angosta de lo que la teníamos. Sondeo con presupuesto sobre un estado de creencias tipado, ediciones de política gateadas por verificador, y pisos de procedencia sobre acciones están **cada uno establecido**. Ninguno de los tres es nuestro para reclamar, y decirlo cuesta menos que que nos lo digan.

Lo que queda es una **conjunción**, enunciada por lo que excluye: una capa de decisión que (a) elige **qué topología de control de flujo correr**, por request, de un catálogo de ellas — ninguno de los tres rutea entre *paradigmas*; (b) puede **abstenerse**, con la curva riesgo–cobertura reportada en vez de la utilidad de lo que eligió contestar; y (c) poda por **aritmética sobre el presupuesto declarado antes de cualquier inferencia**. Sacando cualquiera de las tres, el resto queda cubierto por el trabajo de arriba.

Y uno de los cuatro nos apoya, desde un lugar al que no llegamos. *Trace2Policy: From Expert Behavior Traces to Self-Evolving Decision Agents* (arXiv 2606.10457) reporta un despliegue en producción de 22 días sobre 3.349 casos resueltos, y encuentra que **a lo largo de cinco escalas de modelo, la varianza atribuible a la versión de la regla supera a la atribuible a la elección de modelo**. Independiente, a escala de producción, y lo más parecido a corroboración externa que tiene esta línea: **la estructura decide más que el modelo**.

5.1 El Teorema del Valor de Selección

Sea p^* el fallback y $p_1 \dots p_k$ los especialistas. Sea $\Delta_j(t) = u(t, p_j) - u(t, p^*)$. Para cada brazo, partir el espacio de tareas **en tres** — la tercera parte no es un tecnicismo, y §5.1.1 muestra qué cuesta colapsarla:

$$\begin{aligned}
S_+^j &= \{ \Delta_j > 0 \} & \text{ganancia estricta} & \quad \pi_j = \Pr[S_+^j] \\
S_0^j &= \{ \Delta_j = 0 \} & \text{empate} & \quad \tau_j = \Pr[S_0^j] \\
S_-^j &= \{ \Delta_j < 0 \} & \text{pérdida estricta} & \quad \nu_j = \Pr[S_-^j]
\end{aligned}$$

y sean, **condicionadas a lo efectivamente ruteado**,

$$\begin{aligned}
\alpha_j &= \Pr[\text{rutear a } p_j \mid S_+^j] & G_j &= E[\Delta_j \mid \text{rutear a } p_j, S_+^j] \\
\beta_j &= \Pr[\text{rutear a } p_j \mid S_-^j] & L_j &= E[-\Delta_j \mid \text{rutear a } p_j, S_-^j]
\end{aligned}$$

Teorema 1. Para todo $k \geq 1$,

$$V(r) - V(p^*) = \sum_j (\pi_j \cdot \alpha_j \cdot G_j - \nu_j \cdot \beta_j \cdot L_j)$$

exactamente, sin ningún supuesto de independencia. Por lo tanto la selección le gana al mejor paradigma fijo si y sólo si $\sum_j \pi_j \alpha_j G_j > \sum_j \nu_j \beta_j L_j$.

Demostración. $V(r) - V(p^*) = E[\Delta_{\{r(t)\}}(t) \cdot 1\{r(t) \neq p^*\}]$. Descomponiendo por destino queda $\sum_j \Pr[\text{rutear a } p_j] \cdot E[\Delta_j \mid \text{rutear a } p_j]$, y partiendo cada esperanza condicional según el signo de Δ_j : la parte de S_+^j pesa $\pi_j \alpha_j$ con media G_j , la de S_-^j pesa $\nu_j \beta_j$ con media $-L_j$, y S_0^j **aporta exactamente cero** porque ahí $\Delta_j = 0$. ■

Dos decisiones lo vuelven exacto y no aproximado. Condicionar G y L a los subconjuntos *ruteados* elimina todo supuesto de independencia entre dónde vive la ganancia y dónde el ruteador elige ir. Y descomponer **por destino** en vez de por un único «especialista» lo hace valer para un catálogo: con k brazos, equivocarse tiene un *destino*, y mandar una tarea a un brazo apenas peor no es el mismo evento que mandarla al peor de doce.

Corolario 1 (precisión sobre cobertura). Cuando p^* es casi óptimo en regiones amplias, G es chico y L grande, así que la condición exige $\beta \rightarrow 0$ incluso a costa de α . El recall no es el objetivo.

5.1.1 Un empate no es un misruteo

S se define con desigualdad *estricta*, así que los empates quedan afuera. Una versión anterior de este teorema definía β sobre el complemento de S_+ , lo que le cobraba al ruteador haber ruteado sobre un empate — un acto que no cuesta absolutamente nada. Construido: un ruteador que rutea sólo donde gana o empata, y nunca donde pierde, registra **$\beta = 0,714$ sin haber hecho un solo daño**.

El producto $\beta \cdot L$ seguía siendo correcto, porque L absorbía el cero. Pero β sola dejaba de ser la *tasa* de misruteo, y el Corolario 2 usa β y L por separado. Definir β sobre S_- restituye la lectura que un lector espera, y deja el Teorema 1 intacto: los empates aportan cero a los dos lados.

No es un caso de borde en ningún catálogo donde varios paradigmas resuelven la misma tarea. En el nuestro, un brazo es el más barato al empatar en 46 de 96 celdas.

5.1.2 El umbral de imposibilidad, y contra qué pérdida se mide

Definir, por brazo,

$\bar{L}_j = E[-\Delta_j \mid S_j]$	la pérdida media de la TAREA	(no depende del ruteador)
$\rho_j = \bar{L}_j / \bar{L}$	la SELECTIVIDAD DE PÉRDIDA	($\rho_j := 1$ cuando $\beta_j = 0$)

Corolario 2. El ruteador le pierde a siempre-fallback si y sólo si $\sum_j \pi_j \alpha_j G_j < \sum_j v_j \beta_j \rho_j \bar{L}_j$, y con un solo brazo

$$\beta_{\max}(\rho) = \pi \cdot \alpha \cdot G / (v \cdot \rho \cdot \bar{L})$$

ρ	qué ruteador es	qué hace el umbral
$\rho = 1$	ciego a la magnitud de la pérdida — se equivoca de forma representativa	el enunciado clásico, y ahí es exacto
$\rho < 1$	evita las equivocaciones caras	se afloja
$\rho > 1$	anti-calibrado: falla justo donde más duele	se endurece

Por qué el parámetro es necesario y no un refinamiento. Enunciado sólo con la pérdida distribucional, el umbral no es una imposibilidad. Alcanzan tres tareas: una ganancia de $+0,10$ con probabilidad $0,10$, ruteada; una pérdida de $-0,001$ con $0,45$, ruteada; y una de $-1,00$ con $0,45$, no ruteada. Entonces $\beta = 0,500$ supera $\beta_{\max} = 0,0222$ por $23\times$ y el ruteador **igual captura $+0,00955$** . Un ruteador que se equivoca *seguido pero barato* es exactamente lo que produce un ruteo selectivo bien construido.

Tampoco alcanza con sustituir por la pérdida realizada: un ruteador perfecto no realiza pérdida, así que el umbral reportaría infinito y parecería no imponer restricción alguna — la objeción que motivó la forma distribucional en primer lugar. **Las dos objeciones son correctas.** Se disuelven juntas en cuanto la pérdida realizada entra como *factor de la pérdida* y no como *denominador de un umbral*: con $\beta = 0$ el término entero se anula antes de que ρ se consulte.

Corolario 2b. Un ruteador obligado a elegir no controla ni β ni ρ . Uno selectivo controla los dos — y ρ es la palanca más barata: bajar β exige acertar más seguido; bajar ρ sólo exige abstenerse donde la apuesta es cara. Eso le da al Corolario 1 un mecanismo y no sólo una desigualdad, y ρ se recupera de cualquier registro que reporte pérdida potencial y realizada.

5.1.3 Cobertura óptima

Sea $v(c)$ el valor capturado a cobertura c , admitida bajando un umbral de confianza. Entonces $dv/dc = E[\Delta \mid \text{tarea marginal en } c]$, y por lo tanto:

(a) v es unimodal **si y sólo si** $c \mapsto E[\Delta \mid \text{marginal en } c]$ es no creciente — es decir, si y sólo si la señal de confianza ordena las tareas por *ganancia esperada*. **La calibración sola no da eso.** La calibración restringe la probabilidad de ganar; el valor depende de su magnitud. Construido, con $\Pr[\text{acertar} \mid \kappa] = \kappa$ exactamente en cada grupo: tres grupos con $E[\Delta]$ de $+0,008$, $-0,170$ y $+0,593$ a confianzas $0,90$, $0,60$ y $0,30$ producen una curva de valor capturado que **sube, baja y vuelve a subir**, con su óptimo en **cobertura total**.

(b) Sin ningún supuesto: la cobertura óptima es < 1 siempre que alguna tarea con $\Delta_{\{r(t)\}}(t) < 0$ fuera ruteada a cobertura total. Esto es lo que el argumento necesita, y se sigue en una línea — sacar un término negativo aumenta la suma.

Corolario 3. Afirmamos (b). Afirmar unimodalidad afirma más de lo que el diseño necesita, y regala un contraejemplo.

5.1.4 La identidad de la brecha del oráculo, y su alcance

Con un solo especialista, la brecha del oráculo iguala $\pi \cdot G$, lo que permite recuperar el β implícito de un ruteador publicado a partir de sus números de portada. **Con k brazos no factoriza:** la brecha es $E[\max_j \Delta_j^+]$, que no es $\pi_j \cdot G_j$ de ningún par fijo. Los 17,1pp reportados para una suite publicada pueden leerse como $\pi \cdot G$ sólo donde ese trabajo reporte un *par*; sobre una grilla, la lectura del β implícito no está disponible.

5.1.5 Qué es el Teorema 1, y qué no es

Es una *identidad*: una descomposición algebraica exacta, verdadera por construcción. **Ninguna medición puede falsarla**, y nada en este paper debe leerse como que la confirmó. Lo empírico es sólo si sus términos satisfacen la desigualdad sobre una distribución dada — una pregunta sobre un ruteador, no sobre el teorema.

El Teorema 1 y los tres corolarios están verificados en `tests/test_science.py` contra distribuciones cuyos términos se conocen por construcción. La identidad se chequeó además sobre 4.000 distribuciones al azar con k de 1 a 4 —3.269 de ellas con empates— con discrepancia máxima $1,67 \times 10^{-16}$. Los Corolarios 2 y 3 están enunciados arriba en su forma corregida; los contraejemplos que forzaron la corrección se reproducen ahí.

Y la distinción importa acá porque los términos nunca se separaron. En los dos corpus held-out el margen de decisión del ruteador fue **0 en todas las tareas**, así que la curva riesgo–cobertura colapsa a un solo punto en el origen: **AURC 0,000** contra un techo de **+0,400**. Un ruteador que nunca se abstiene no tiene α ni β distintos de siempre-fallback, así que la identidad se cumple vacuamente — con los dos términos medidos sobre una cobertura que el ruteador no eligió.

El trabajo que un lector podría acreditarle a §5.1 lo hace §5.2. El teorema de selección aporta la contabilidad; toda afirmación falsable que este registro llegó a resolver es sobre dominancia de cascada y sensibilidad del detector. Presentarlos en este orden es una decisión de exposición, no una de prioridad — y la lectura honesta es que la rama de selección de la partición de más abajo **sigue sin ejercitarse**.

Una observación une las tres correcciones. π , α y β responden *si* el ruteador acierta; G , L y p responden *cuánto cuesta cuando no*. Cada lugar donde el enunciado anterior falló —el umbral, los empates, la unimodalidad— es un lugar donde esos dos ejes se trataron como uno.

5.2 Dominancia de la cascada, y su corrección medida

Un **ruteador** que se equivoca paga L , una pérdida de calidad: se entrega una respuesta peor y nada la recupera. Una **cascada** que se equivoca paga $\text{cost}(p_1)$, una pérdida de costo: el intento barato se desperdicia y la buena respuesta llega de todas formas tras escalar.

Con sensibilidad de detector s :

```
regret(ruteador) =  $v \cdot \beta \cdot p \cdot L$   
regret(cascada) =  $E[\text{costo de la escalera}] + (1-s) \cdot L$ 
```

Una corrección que le debemos a la medición. Un enunciado anterior de este resultado daba el término de costo como $\text{cost}(p_1)$. Es falso. Cuando ningún escalón satisface al detector, la cascada corre la escalera *completa*: en nuestro estudio sintético capturó el 100% de la brecha a **4,4x** el costo del mejor paradigma fijo. El límite es el costo esperado de la escalera, y el patrón requiere un tope de presupuesto.

La sensibilidad del detector es la variable crítica, y un detector débil es catastrófico y no simplemente subóptimo. Con $s = 0,6$ la fracción capturada midió **-5,98**: una falla no detectada deja a la cascada detenida en un escalón barato con una respuesta mala. Es el análogo estructural del Corolario 2 — como un ruteador con β alto pierde, una cascada con s bajo pierde, y pierde más fuerte.

Corolario (la partición por verificabilidad).

```
 $v = 1$  (existe un oráculo barato) -> cascada; no hace falta ruteador  
 $v = 0$  (no hay oráculo) -> rutear, bajo la disciplina del Teorema 1
```

Predecir sólo es necesario donde verificar es imposible. Si esto se sostiene, buena parte de la literatura de ruteo está resolviendo el problema equivocado en el régimen verificable.

Y la partición tiene una consecuencia metodológica filosa que pagamos por aprender. La rama $v = 0$ es la que necesita un ruteador, y es precisamente la rama que un benchmark de coincidencia exacta no puede contener: el gold es lo que hace que corregir no necesite juez, y el gold es un detector. En este registro la confluencia fue literal —un mismo campo servía para las dos cosas— y el resultado es que dos predicciones de ruteo preregistradas, sobre dos corpus held-out independientes, las contestó la regla de cascada mientras la de selección no se evaluó nunca. Los números son reales; lo que miden es la rama $v = 1$.

Separar las dos no es un cambio de calibración. Requiere un corpus que **declare la disponibilidad de detector por tarea** sobre bases independientes de la clave de respuestas —para nosotros, si verificar es más barato que resolver— y mueve la cascada de disparar en 22 de 26 tareas a disparar en 2.

5.3 Soundness del ensamblador

Los tres resultados que siguen son **estructurales**: ninguno depende de un corpus, de un modelo ni de una corrida. Están acá porque la maquinaria que describe §6 existe para hacer posibles enunciados de esta forma, y sin ellos la procedencia es contabilidad — se registra, se muestra, y no compra nada que alguien pueda nombrar.

Sea T una plantilla con ranuras $S = \{s_1 \dots s_n\}$, B una asignación de ranuras a proposiciones, Γ una base de creencias y ϕ un piso de procedencia.

Teorema 2 (soundness del ensamblador). Si $\text{fill}(T, B, \Gamma, \phi)$ emite una cadena R , entonces para toda ranura $s_i \in S$ existe $\beta_i \in \Gamma$ tal que

1. β_i es la creencia **vigente** sobre la proposición que B asigna a s_i ;

2. `rank(procedencia( $\beta_i$ )) \geq rank(\phi)` ;
3. la subcadena de `R` en la posición de `si` es exactamente `str(valor( $\beta_i$ ))` .

Y `R` no contiene ninguna subcadena que provenga de otro lado que no sea `T` o esos valores.

En una línea: **si el ensamblador emite, todo número que emitió está implicado por la base de creencias al piso pedido**. No «probablemente». No «salvo alucinación».

Demostración. Por construcción, y depende de una línea. `fill` recorre `slots_of(T)` —las ranuras que la plantilla realmente usa, extraídas de ella y no declaradas aparte— y para cada una hace exactamente tres cosas: resuelve la proposición asignada, pide la creencia vigente y compara el rango de procedencia contra el piso. Cualquier fallo anota el motivo y sigue, sin emitir. La sustitución ocurre **después** de comprobar que la lista de rechazos está vacía, así que `values` tiene una entrada por cada ranura de `T`, todas provenientes de una creencia vigente que pasó el piso; y la sustitución reemplaza ocurrencias del patrón de ranura dejando intacto el resto de la plantilla, que es texto fijo escrito por el código. Las tres condiciones se corresponden una a una con las tres guardas, y no hay un cuarto camino por el que un valor llegue a la salida. ■

Falla cerrada y falla ENTERA, que es una decisión y no una consecuencia. Si una sola ranura no llega al piso, no se emite una versión parcial. Emitir «*el saldo de la cuenta es ___*» no es más honesto que emitir un número inventado: es el mismo acto con mejor caligrafía, e invita al lector a completar lo que el contrato rechazó.

Cuatro límites, dichos adentro del teorema y no en una nota al pie.

límite	qué significa
el alcance es la ranura, no la oración	« <i>el saldo NO supera {x}</i> » con <code>x</code> correcto es sound y falso . No es un defecto de la implementación: es la frontera de la familia entera, y un red-team la mide en vez de suponerla
la procedencia es del registro, no del mundo	<code>COMPUTED</code> significa que alguien la computó y la asentó. El teorema traslada confianza desde el piso hacia la salida; no la crea. Un sensor que miente se emite con procedencia impecable
vigente, no histórica	la garantía es sobre el estado de creencias al ensamblar , no sobre todo lo que alguna vez se creyó
<code>str()</code> es parte del teorema	la condición 3 dice <code>str(valor)</code> , no «el valor». El ensamblador no formatea, porque formatear sería empezar a decidir algo sobre el número

Verificado exhaustivamente y no por casos elegidos: sobre el producto cartesiano de las cuatro procedencias por las condiciones de rechazo — un espacio chico, y por eso uno que se puede recorrer entero.

5.4 La cota nativa del ratchet

El piso de garantía aprendido sólo sube. Un borrador anterior lo acotaba importando un teorema sobre **varianza bajo oscilación**; eso no era una cita floja sino un **error de categoría** — una secuencia monótona y acotada tiene varianza que tiende a cero por construcción, así que la cota se cumplía

vacuamente y no decía nada. Lo que un ratchet necesita que se le acote no es cuánto **oscila** sino cuánto **daño acumulado** puede hacer antes de detenerse, y eso es un conteo.

Los niveles son `EXPLORATORY < STANDARD < ACCOUNTABLE < CERTIFIED`, y el techo aprendido es el tercero: `CERTIFIED` queda fuera de alcance a propósito, porque ese nivel restringe qué patrones son admisibles y una estadística sobre calidad de evidencia no es evidencia sobre certificabilidad.

Proposición 4 (daño total acotado). Sobre R regiones, el número total de eventos de endurecimiento en **toda la vida del sistema** es $\leq 2R$, sea cual sea la cantidad de ciclos de consolidación.

Demostración. Monotonía: el piso de cada región es una secuencia no decreciente en un conjunto finito y totalmente ordenado, así que cambia a lo sumo tantas veces como niveles haya por encima de su base. No hace falta nada probabilístico. ■

Proposición 5 (la guarda de replicación es fuerte lejos del umbral y débil cerca). Con q la tasa verdadera de rechazo de la región y dos splits de tareas **disjuntos**:

q	un split	ambos	$\approx$
0,10	0,0050	0,000025	1 en 39.613
0,25	0,1138	0,01295	1 en 77
0,40	0,4059	0,16477	1 en 6
0,45	0,5230	0,27358	1 en 4

Eso se dice y no se esconde, e importa menos de lo que parece por dos razones estructurales. Cerca del umbral un falso positivo es casi indistinguible de un verdadero —una región cuyo q real es 0,45 **efectivamente** rechaza casi la mitad de las veces—. Y la Proposición 4 acota el daño acumulado pase lo que pase.

La monotonía que vuelve inaplicable el teorema de varianza prestado es exactamente lo que acota el daño de su propia tasa de falsos positivos. La propiedad que rompe la cota prestada es la que la hace innecesaria.

Y lo que cuesta está medido, lo cual corrige cómo se lee la Proposición 4.

nivel	brazos admisibles	cobertura	u (mejor fijo)
$A0 \cdot A1 \cdot A2$	5	100%	0,6101
A3	2	40%	0,4221

El ratchet es **gratis hasta A2 y cuesta todo de una vez en A3**: la única transición con precio se lleva **60% del catálogo y 31% de la utilidad**. «A lo sumo dos subidas» invita a imaginar un daño que se acumula despacio; lo medido es lo contrario — **una sola transición tiene precio, y ahí es abrupto**. Las otras dos son gratis porque no hacen nada. Y el promedio esconde a quien paga: una región pierde **−0,5000** mientras la media del corpus es 0,0000.

5.5 Quién fija el dial

La pregunta parece de gobernanza y es de diseño. Si el nivel de garantía lo elige el llamador, un llamador apurado lo baja; si lo elige el sistema, el llamador no puede pedir más rigor del que el sistema cree necesario. **Ninguna de las dos.**

```
nivel efectivo = max( pedido , piso de creencias , piso aprendido )
```

fuelle	quién la produce	qué puede hacer
pedido	el llamador, en el request	subir, nunca bajar
piso de creencias	required_floor(Γ) sobre las creencias COMPUTED del request	subir, y no se puede desactivar
piso aprendido	θ.floors[región] , dentro del bundle firmado	subir, y sólo si está promovido

Proposición 6. `max` es la **única** composición bajo la cual cada fuente sólo puede endurecer. Con `min` o con un promedio, agregar una fuente podría ablandar el resultado — y entonces una fuente nueva sería un **riesgo** en vez de una garantía.

Por qué el llamador sube y no baja. Conoce cosas que el sistema no: que este request va a un informe regulatorio, que el resultado se publica, que hay un auditor mirando. Nada de eso está en el material. Lo que no puede es pedir **menos**, porque el piso sale de propiedades **del request mismo** — irreversible eleva a A3, `shared_writes` a A2, y las dos entran como creencias `COMPUTED` **declaradas por el caller, nunca inferidas del texto**. Un llamador que pudiera bajar el piso podría declarar una acción irreversible y después pedir tratarla como exploratoria, que es exactamente la combinación que el piso existe para impedir.

Una cuarta fuente, que no es un nivel sino una degradación. A2 admite creencias `ELICITED` , pero sólo una vez que la calibración se ganó; admitirlas antes anula el propósito del nivel. Así que la resolución no baja el nivel: **endurece el piso de procedencia dentro del nivel**. La misma idea del `max` , aplicada al otro eje.

Y se evalúa marginalizando sobre sus posiciones, no fijando una — reportar métricas a un dial fijo reporta una política, no un sistema. Marginalizar produjo un hallazgo sobre el propio dial:

Tres de las cuatro posiciones son indistinguibles. A0, A1 y A2 declaran `admissible_patterns = None` , así que **el dial no restringe el catálogo hasta A3**. Dos de sus tres transiciones no hacen nada en esa dimensión, y toda la diferencia se paga en un solo escalón.

Lo que sí distingue A1 de A2 vive en otros ejes —θ firmada, log de creencias, profundidad de composición, y el piso de procedencia—, así que el dial no es inerte ahí: es inerte **en la dimensión que esa tabla mide**. Decir cuál es cuál es el punto de marginalizar.

5.6 Lo que la teoría no supone, y por qué eso es la afirmación

Todo lo de §5.1–§5.5 está enunciado sobre un catálogo de brazos, una utilidad, una base de creencias y un retículo de procedencia. **Ninguno de los cinco resultados menciona recuperación, documentos ni respuesta a preguntas.** No es un accidente de redacción ni una salvedad: es la afirmación. Lo que se

describe es una capa de decisión sobre *acciones que el sistema puede tomar*, y el ruteo de paradigmas sobre un corpus documental es la instancia que pudimos medir sin juez.

La distinción sobre la que la capa realmente gira no es *qué clase de tarea* sino **qué se sabe y cuándo**. Una regla sólo puede gobernar si se la puede evaluar al momento de decidir; un valor sólo se puede emitir si una creencia vigente lo lleva a la procedencia exigida; un nivel sólo se puede subir. Las tres son propiedades de la decisión, no del dominio.

La misma maquinaria, enunciada sobre cuatro superficies. Los teoremas de arriba son la forma general; las columnas son lo que instanciarlos exige.

superficie	el sensor emite	la regla decide	el registro guarda
contenido	números con procedencia	emitir o negarse, por ranura (§5.3)	qué creencia llenó qué ranura
datos	una consulta propuesta, su grano, su resolución temporal	admitir la consulta o exigir elicitación	la consulta, y contra qué se la chequeó
acciones	una llamada a herramienta y sus precondiciones	el piso de procedencia sobre lo irreversible (§5.5)	un ledger de idempotencia
gobierno	una edición candidata de la política	la guarda de promoción sobre episodios held-out	el diff entre dos bundles firmados

La selección entre topologías de control es la **primera** columna instanciada sobre un catálogo de recuperación. Es el caso que medimos, no el alcance de lo que se afirma.

Y la rama no probada de la teoría y la superficie no medida son el mismo lugar. §5.2 parte el problema sobre v , la disponibilidad de un detector barato, y §1.3 registra que todos los corpus de acá caen del lado $v = 1$ **por construcción**, porque el gold es lo que vuelve la calificación libre de juez y el gold es un detector. Así que la rama $v = 0$ —la que necesita un router— no la puede alcanzar ningún benchmark que califique por exact-match.

¿Dónde está $v = 0$, entonces? Predominantemente en la superficie de acciones. Chequear *si un archivo se escribió* es barato; chequear *si éste era el reembolso correcto* no lo es, y no hay clave de respuestas que lo abarate. El dominio que este registro no mide es el dominio donde la partición central de la teoría por fin tiene dos lados.

Así que la ambición se enuncia en vez de matizarse. La teoría es general por construcción y está verificada como tal —sobre distribuciones sintéticas con respuesta conocida, no sobre corpus—. Las mediciones son sólo de recuperación, y §9 dice exactamente qué herramientas existieron y cuáles nunca. Un lector debería tomar §5 como afirmado para agentes en general y §7–§8 como afirmado para extracción de respuesta exacta, y exigirnos la brecha entre las dos en vez de una promesa más angosta que no hicimos.

6. Diseño

6.1 Medir sin juez

Cada tarea trae un oráculo de conjunto, así que la calidad es F1 de conjuntos tras normalización. §2.7 da la razón: el efecto es de un dígito de puntos porcentuales y la varianza del juez es del mismo orden, así que un juez no agregaría solamente ruido sino un **sesgo** — el sesgo de verbosidad favorece las respuestas largas que producen los paradigmas caros, que es precisamente la comparación bajo prueba.

Un oráculo vacío es una pregunta legítima e importante — *listá todos los X* donde no hay ningún X — y testea si un paradigma inventa items. Se califica exigiendo un enunciado explícito de vacuidad; el silencio saca cero, porque un paradigma que no devolvió nada porque se cayó no debe puntuar igual que uno que buscó y reportó no haber encontrado nada.

6.2 Creencias, procedencia y garantía por solicitud

El determinismo pleno no está disponible con un LLM, y perseguirlo excluyendo al modelo de la decisión descarta información que el modelo tiene. La inversión es de HADD (§2.4): el modelo es un **sensor** que emite proposiciones tipadas, una capa simbólica determinista decide sobre la base de creencias, y la garantía toma la única forma que puede tener:

no "el mismo prompt da la misma respuesta" — falso, siempre sino "la misma base de creencias da la misma decisión" — la estabilidad de decisión de HADD, con la base registrada

Lo que esta sección agrega es la **epistemología de la creencia misma**. En HADD la procedencia es un campo de trazabilidad; acá carga el peso de la decisión como jerarquía tipada: `COMPUTED` (una función pura, credencia 1,0) > `OBSERVED` (medido ejecutando una sonda) > `ELICITED` (el modelo lo afirmó, sujeto a calibración) > `ASSUMED`. Una regla puede exigir una procedencia mínima, así que una acción irreversible puede restringirse a evidencia computada y observada: *la opinión de un modelo de que una acción es segura no es evidencia admisible para tomarla*.

La garantía es entonces una propiedad **de la solicitud**, no del sistema. Un modo global hace que todo el tráfico pague por la solicitud más estricta. Cuatro niveles graduán el piso de procedencia, si θ puede aprender online, si la corrida es reproducible desde caché, y **qué patrones son admisibles** — el nivel certificado excluye topologías cuyo flujo de control es no acotado, no porque sean peores (a menudo son mejores) sino porque sus modos de falla no son enumerables. El piso se deriva de creencias sobre la solicitud: quien llama puede pedir más y nunca menos. La derivación misma aprende, offline: una categoría de solicitudes cuyas afirmaciones elicítadas son rechazadas repetidamente por la compuerta es una categoría cuyo piso sube — las estadísticas de rechazo son evidencia sobre la clase de solicitud, y consumirlas cierra el bucle sin ajustar jamás nada adentro de una solicitud.

Tres propiedades lo vuelven seguro de correr, y cada una está impuesta, no pretendida.

El evento es tipado, no parseado. Un rechazo lleva su motivo como valor — la procedencia que se tenía contra la que la regla exigía — de modo que la estadística cuenta el evento. Contarlo emparejando

subcadenas de la explicación mediría el fraseo, y el fraseo es prosa que se reescribe. Sólo cuentan los rechazos por PROCEDENCIA insuficiente: una creencia rechazada por credencia baja, o por tener el valor equivocado, es el sistema funcionando, y no dice nada sobre el régimen de evidencia de la clase.

La guarda es replicación, no utilidad. El registro se parte por tarea; una mitad propone las regiones cuyo piso debería subir, y el piso se instala sólo si la otra mitad —solicitudes que la propuesta nunca vio— dice lo mismo de manera independiente. La utilidad sería el criterio equivocado acá, y descartarla no es una concesión: subir un piso hace que el sistema exija evidencia medida donde habría actuado sobre una afirmación, lo que cuesta tokens y sólo puede bajar la utilidad medida en el corto plazo. Un piso de gobierno puntuado por la utilidad que produce es un piso que nunca sube.

El aumento es acotado y monótono. Se detiene en accountable y nunca llega a certified, porque certified además restringe qué patrones pueden correr y una estadística sobre calidad de evidencia no es evidencia sobre certificabilidad — un piso que aprende no puede quedar habilitado a descalificar una topología. Y nunca baja solo: la ausencia de rechazos después de que un piso sube es justamente lo que ese piso se instaló para producir, así que leer esa ausencia como motivo para bajarlo sería una oscilación puesta en el diseño.

Los pisos aprendidos viajan en el bundle de política firmado, así que nada puede elevar el nivel de una solicitud salvo por el mismo camino de promoción que recorre θ . Verificado: una región cuyos rechazos replican a través de la partición recibe su piso; una región que califica sólo en la mitad que la propuso, no.

Credencia y tamaño de efecto no deben confundirse. Un margen aprendido es una creencia *certera* sobre un efecto *grande* — credencia 1,0, valor 0,9 — y codificar la magnitud como credencia reporta un hecho computado como incierto, destruyendo la distinción para la cual existe la capa.

6.3 Consolidación de la política de control

El aprendizaje es offline y copy-on-write. Los episodios se reproducen en orden de **sorpresa** y no cronológico, lo que elimina el sesgo de recencia que la actualización online tiene por construcción; las estadísticas se reducen en escala y las entradas sin uso se podan; y una etapa de abstracción busca particiones de features que separen paradigmas mejor que el binning actual.

Dos disciplinas hacen esto seguro. Una política candidata se instala sólo si no regresa sobre episodios retenidos. Y el registro se parte en tres **por tarea** — una parte propone particiones, una las puntúa, una la toca sólo el guardia de promoción — porque buscar muchas particiones contra un solo holdout es la forma en que detectar una verdad se vuelve confabularla. Una partición descubierta entra con cero episodios, por debajo del piso de confianza, y no puede gobernar una decisión hasta haber ganado evidencia: **un sueño es una hipótesis, no un hecho.**

Verificado: dado un registro donde la utilidad es independiente de todo atributo, ninguna partición sobrevive la validación.

6.4 La superficie de herramientas es parte de la topología

Elegir *cuál* modalidad de recuperación, a *qué* granularidad, en *qué* secuencia, con cuánto *batching* es asunto propio del agente — **es** la topología. Un harness que ofrece una búsqueda tosca y un read ya decidió las cuatro cosas y después mide lo que queda.

Cuatro herramientas en tres granularidades, con lo léxico y lo denso expuestos por separado junto al punto de entrada fusionado:

herramienta	devuelve	tamaño medido, 3 unidades
search	resúmenes	312 chars
keyword_search	highlights << >>	937
semantic_search	texto completo	3.502
read	texto completo, en batch	3.501

11× entre resumir y leer, y esa diferencia es invisible cuando toda búsqueda devuelve un extracto de tamaño fijo. Fusionar léxico y denso en una sola herramienta híbrida fue un error concreto: híbrido es el default correcto, pero exponer sólo la vista fusionada significa que el agente nunca puede pedir coincidencia exacta para un identificador — y las respuestas de dos de nuestras celdas *son* identificadores.

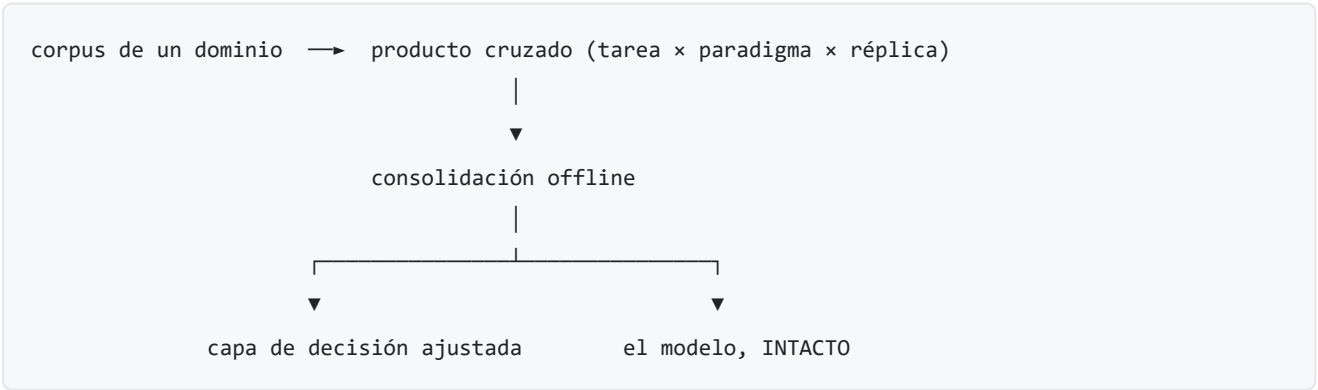
La calidad de recuperación está **implementada** como una variable con nueve brazos: híbrido (BM25 + denso, fusión RRF), HyDE, un híbrido con rerank, léxico, semántico, tres simulaciones degradadas a recall y precisión declarados, y un oráculo. Las simulaciones son funciones deterministas de (tarea, consulta, unidad) , así que la calidad de recuperación **puede ser** un dial controlado y no otra fuente de ruido.

Toda medición de este paper la mantiene fija en híbrido, así que los resultados de abajo son resultados en un punto de ese dial. La única observación desde otro punto —el denso solo ganándole al híbrido en la celda acoplada— salió de una sonda, no de la grilla. Los vectores se cachean por hash de contenido, así que después de una primera pasada la fusión es aritmética local.

Un hallazgo contra la visión recibida: en la celda acoplada, denso solo midió recall 0,75, híbrido 0,50, léxico 0,25. RRF promedia rangos, así que un componente que anda mal arrastra a uno que anda bien. "Híbrido siempre es mejor" no se sostiene cuando un componente está muy por debajo del otro.

6.5 El banco es un procedimiento de ajuste, no sólo un instrumento

El modelo está **congelado y no se entera**. Lo que se ajusta es la capa de decisión, y se ajusta **sobre el registro del producto cruzado**: cero llamadas nuevas, replay contrafáctico sobre filas ya pagadas.



Eso le da al banco una segunda lectura que el resto de este paper no usa: no es sólo el instrumento que mide los paradigmas, es **el procedimiento de ajuste de la capa que los elige**. El mismo producto cruzado que produce el número produce la política.

Un experimento de aprendizaje automático donde el aprendiz no es el modelo. No hay gradiente; hay estadísticas por región sobre episodios registrados, con guarda de promoción. Y por eso el artefacto ajustado es **legible y diffeable** — dos versiones de la política se comparan como código, no como pesos.

Qué se ajusta, exactamente — el inventario sale del bundle firmado y del ciclo de consolidación, no de una intención, y dos filas son el punto:

se ajusta	qué es	estado
stats	por (región, paradigma) : tasa de victorias y peso Hebbiano	ejecutado ; el peso no lo lee el router — está probado que no puede mejorar un argmax
floors	el piso de garantía por región, aprendido de estadísticas de rechazo tipado	ejecutado , con guarda de replicación y techo en ACCOUNTABLE (\$5.4)
model_stats	ídem por (región, modelo) : la segunda política	ejecutado
trusts_elicited	calibración de la credencia elicitada, computada desde el log de creencias	ejecutado , y viaja firmado adentro del bundle , no como parámetro
clauses	cláusulas de adquisición certificadas	ejecutado ; hoy ninguna se promueve — el beneficio neto no supera el piso de ruido al λ de decisión
asociaciones de orden	pares ordenados de herramientas, reforzados por resultado	medido ($p = 0,0078$) y ningún consumidor lo lee
el reparto del handoff	qué unidades ve cada sub-agente	NO se ajusta : es un paso por índice, fijo

Las dos últimas filas son donde el aprendizaje existe como medición y todavía no como mecanismo. Decirlo vale más que insinuar que ya funcionan.

Cuatro disciplinas lo vuelven un ajuste y no una ilusión. Un episodio es una *celda*, no una réplica — contarlas por separado es pseudorreplicación, y computar «fue el mejor» sobre trials crudos deja que una réplica con suerte cobre el refuerzo que la media de su paradigma nunca ganó. El registro se parte en tres *por tarea* (§6.3), y el mundo final es de un solo uso, gastado *antes* de responder. Y los ejes de partición están **tipados por cuándo se conocen**: una regla sólo puede gobernar si se la puede evaluar al momento de decidir, así que partir sobre `truth_coupling` —el oráculo del extractor— o sobre `iterations` —posterior a la ejecución— descubre una regla que no se puede aplicar. El tipo lo dice; no se descubre al cablearla.

****Y la cuarta es lo que *no* entra. Una fila que no es una medición no puede volverse un episodio, y las clases son distintas: un 429 agotado es infraestructura; una celda podada por aritmética no ejecutó, así que el 0,0 que lleva es un relleno y no una lectura; una fila sin región no tiene bin donde ser aprendida. Medido sobre la campaña actual, 39 de 180 episodios —21,7%— venían de celdas que nunca corrieron, y el sesgo no es aleatorio: cae sobre los brazos caros, que son justo los que la poda alcanza. Dos pares reportaban $u = 0,667$ midiendo 1,000 donde efectivamente ejecutaron, y quince pares llevaban $u = 0,000$ con $n = 2-3$ sin una sola ejecución detrás del número**.**

Y el conteo es peor que el promedio. Los episodios son lo que cruza el piso de confianza, así que un par podía **ganar confianza con celdas donde su brazo nunca corrió**. Ninguno había cruzado todavía —la campaña es joven— pero quince estaban en camino, con ceros que ninguna ejecución respalda. La infactibilidad no se pierde: la consume su consumidor, que es el portón de factibilidad, y lo hace *antes* de seleccionar; meterla además en la política cuenta un mismo hecho dos veces, en un canal que no lo sabe representar.

Y el error simétrico importa igual. Una respuesta equivocada, una respuesta vacía de un brazo que *sí* corrió, y un `Unknown` literal del modelo son todas **mediciones** — el brazo ejecutó y falló, que es exactamente lo que la política tiene que aprender. Un filtro que descartara también los fracasos dejaría una política entrenada sólo con éxitos, que es la forma más rápida de aprender que todo funciona.

La afirmación de transferencia, y su refutación medida

La lectura se completa con un condicional: *si el corpus es representativo del dominio, la capa ajustada debería transferir*. Eso es falsable, y este registro ya lo falsó una vez.

P15. Sobre un mundo que la política nunca había visto —seed 47, 390 celdas, cero errores de infraestructura— el ruteo por request perdió contra el mejor paradigma fijo por **-0,087**, más allá del piso de ruido, **mientras reproducía cada decisión 26/26** desde su base de creencias registrada.

El mecanismo es el hallazgo, no el número. El vocabulario de región **no tiene eje de horizonte**, así que las tareas que castigan una elección fija eran indistinguibles de las que la premian. La política ruteó contra su propio veredicto registrado porque **ninguna etiqueta le dijo nunca en qué caso estaba**. Chequeo de sensibilidad: reparar la validez del aprendizaje —agregación por episodio, holdout limpio— deja el número idéntico, así que la refutación no es un artefacto del procedimiento.

Eso afila la condición hasta volverla chequeable:

La representatividad hay que enunciarla sobre los ejes que el vocabulario de región distingue. «Representativo del dominio» no alcanza. Un corpus que varía en una dimensión que el mapa de features no mira produce episodios que la política no puede separar — y entonces aprende un promedio sobre dos poblaciones. La condición se puede chequear sobre un corpus *antes* de correrlo, porque la región es una función determinista de los features.

Una consecuencia que se puede afirmar hoy

Más inferencia no compra más ajuste. La consolidación es replay sobre el registro, así que el costo de aprender es **cero llamadas**: lo que la cuota compra son *episodios*, y el ajuste es gratis sobre los que haya. Eso separa dos decisiones que se suelen tomar juntas — cuánto medir lo gobierna la potencia estadística, cuánto entrenar no lo gobierna nada.

Y es observable mientras una campaña corre. Sobre la campaña actual, **516 filas dan 141 episodios sobre 6 regiones, 57 pares (región, paradigma) con evidencia y 27 con $n \geq 3$** — leído del registro entre dos tandas, sin costo adicional. La potencia estadística la fija el **corpus**, no la cuota: un par junta un episodio *por tarea* de su región, así que cruzar el piso de evidencia exige esa cantidad de tareas. Gastar más agrega tareas, y sólo cuentan si caen en la región justa.

7. Mediciones preliminares

Todos los números de §7 y §8 son **n=1 por celda**, sobre un corpus sintético, con un modelo (gpt-5-chat), bajo recuperación híbrida. La varianza entre réplicas sobre la topología DAG se midió en hasta 3,93× en costo. Las magnitudes son indicativas; los mecanismos descansan en trazas de uso de herramientas y son más robustos.

7.1 El agregado sobre cuatro celdas de features

paradigma	calidad media	tokens totales	rango de costo	unidades alucinadas
Direct	0,938	46.101	1,2×	0
CoT	0,938	46.272	1,2×	0
ReAct	0,688	116.524	27,6×	0
Reflection	0,688	126.254	6,9×	0
Map-Reduce	0,438	62.321	1,2×	0
DAG	0,438	439.319	24,5×	0
Plan-Execute	0,250	123.175	2,0×	78

El paradigma más simple gana en calidad y es simultáneamente el más barato; el más elaborado devuelve menos de la mitad de la calidad por 9,5× el costo.

Esto está gobernado por una amenaza de validez que enunciamos en lugar de enterrar: en este corpus toda tarea entra en un prompt, y en ese régimen leer todo es óptimo y el orden es casi tautológico. §4 existe para escapar de eso, y §9 registra que la escapatoria está construida pero no medida.

7.2 La calidad de las herramientas gobierna la varianza que se le atribuye a la topología

grupo	rango de costo
no usan herramientas (Direct, CoT, Map-Reduce)	1,2×
usan herramientas (ReAct, Reflection, Plan-Execute, DAG)	50×

La calidad de las herramientas no mueve la media; gobierna la varianza — y de forma asimétrica. Los lectores son caros y estables; los buscadores son baratos-o-ruinosos, y qué cara sale de la moneda lo decide la recuperación. Un paradigma que cuesta entre 2.877 y 251.374 tokens según si la búsqueda funcionó no es algo para lo que se pueda presupuestar.

Implicancia: un benchmark que no reporta la calidad de su superficie de herramientas no está midiendo topologías. Está midiendo su retrieveer envuelto de siete maneras.

7.3 Un A/B sobre tres señales de contabilidad, y lo que una réplica le hizo

Mismo corpus, mismo retriever, mismas cuatro celdas; lo único distinto es la superficie de herramientas, registrada como variable por fila.

Los tres paradigmas que no usan herramientas fueron **idénticos al token** entre brazos. Ese control es lo que hace atribuible el resto; sin él cualquier diferencia podría ser ruido de corrida.

Un tercer brazo agregó cuatro herramientas de memoria de trabajo (`note` , `notes` , `plan` , `advance`) y produjo un resultado que no habíamos planeado. **El modelo apenas las llamó:** sobre 28 filas, un `note`, una compactación y cero planes. En un corpus donde todo entra en un prompt no hay presión de contexto, así que una herramienta de memoria de trabajo no tiene trabajo — **exponer una capacidad no es lo mismo que proveerla.**

Ese brazo es entonces un resultado nulo sobre las herramientas a `n` chico —28 filas, en el corpus donde la memoria de trabajo tiene menos que hacer— y algo más útil: una **réplica** accidental de la superficie de contabilidad.

Como réplica dice lo que el agregado esconde. **La reproducibilidad es por tarea, no global.** En tres de las cuatro celdas, 10 de 10 valores de calidad son idénticos entre el par. En la cuarta — la cadena de tres saltos — 2 de 4 se dieron vuelta, y ambas por la unidad completa. La inestabilidad no está repartida finamente sobre el estudio; está concentrada en la tarea más difícil, donde el resultado es bimodal. El costo no fue reproducible en ninguna parte: dispersión media 2,05×, máxima 5,43× a calidad *idéntica*.

Eso convierte al A/B de una lista de efectos en una lista de efectos **con un test de atribución**. Un efecto que mueve sólo la celda que sale cara o cruz no es atribuible; uno que mueve una celda reproducible, sí.

paradigma	Δ reportado	celdas que movió	atribuible	veredicto
DAG	+0,500	c3 (inestable), c5 (estable)	+0,250	sobrevive la mitad
Reflection	+0,250	sólo c3 (inestable)	0,000	retirado
ReAct	+0,000	ninguna	0,000	nulo, confirmado
Plan-Execute	-0,139	c2, c5 (ambas estables)	-0,139	sobrevive

Retiramos el resultado de Reflection. Salió íntegro de la celda que se da vuelta sola, y con $n=1$ no se distingue del ruido.

El resultado de costo del DAG sobrevive, y es lo más fuerte que hay acá. El costo total cayó **3,05×** contra una dispersión de réplica de 1,62× sobre la misma medida, y cayó exactamente donde el mecanismo lo predice: las dos celdas desbocadas, 251k → 51k y 143k → 10k, ambas con avisos de estancamiento. Las otras dos celdas no cayeron nada. Así que decirle a un bucle iterativo que su retriever dejó de producir vale una reducción de costo de 3×, con el mecanismo visible en la traza — mientras la ganancia de calidad que lo acompañaba es la mitad de lo que reportamos primero.

La degradación de Plan-Execute sobrevive, y sobre celdas reproducibles: +0,444 en una y –1,000 completo en otra. Nuestro propio análisis había llamado a la contabilidad de cobertura "la adición de mayor apalancamiento" para este paradigma. La empeoró. Saber que la respuesta está incompleta es inútil cuando la arquitectura no puede completarla: los sub-agentes ya terminaron cuando la síntesis se entera de que faltan unidades. **Un diagnóstico sin capacidad de actuar es peor que ninguno** — y es el único efecto del A/B que movió celdas estables y contradijo nuestra predicción.

Un efecto de segundo orden que no previmos. `read_all` redujo el costo de una lectura completa 7,6× y **subió** el costo neto de ReAct un 27%: abarató el *acto*, y por lo tanto la *decisión*, de leer todo.

Una celda no corrió. Dos filas registraron cero tokens y respuesta vacía — no una respuesta equivocada sino una ausente. Están excluidas de toda cifra de arriba en lugar de puntuadas como cero, porque un paradigma que nunca ejecutó no debe mezclarse con uno que respondió mal.

7.4 El estudio de dos regímenes, con réplicas

Segundo estudio, corrido el 2026-08-26 con las predicciones P1–P5 registradas antes (README del harness). Sets de tareas emparejados sobre `gold_v2` (en ventana, ~18k tokens) y `gold_deep` (fuera de ventana, ~483k), `repeat = 3`, estabilidad por celda. 126 + 158 filas válidas. Dos celdas pesadas fuera de ventana (`react` / `reflection` / `dag` en c2-w48 y c5-w48) tienen menos trials que el resto; sus medias van marcadas † y hay que leerlas con esa `n` más chica.

Primero el barrido de factibilidad, porque no cuesta nada: a 135k tokens la aritmética poda leer-todo en 12 de 32 tareas; a 483k en **24 de 26**; a 1,27M en 24 de 32 — y a esa escala la poda alcanza a `map_reduce` en 18–20 tareas, cuyo gasto proyectado excede el presupuesto declarado (P1, confirmada). En ventana no se poda nada. El claim de régimen es aritmética, no medición.

En ventana (utilidad media / tokens medios por celda; flips = celdas cuya utilidad cambió entre réplicas):

paradigma	C2	C3	C4	C5	flips	costo mediana (rango)
<code>direct</code>	0,75 / 13k	1,00 / 11k	1,00 / 13k	1,00 / 11k	0/6	10,5k (1×)
<code>react</code>	0,75 / 25k	0,67 / 63k	1,00 / 3k	1,00 / 61k	3/6	23,8k (70×)
<code>map_reduce</code>	0,75 / 16k	0,00 / 14k	1,00 / 16k	0,00 / 16k	0/6	15,1k (1×)
<code>plan_execute</code>	0,00 / 27k	0,00 / 11k	0,00 / 32k	0,33 / 29k	1/6	19,0k (12×)
<code>reflection</code>	0,83 / 31k	0,44 / 40k	1,00 / 8k	0,67 / 113k	3/6	21,2k (27×)
<code>dag_strategy</code>	0,75 / 27k	0,33 / 77k	1,00 / 6k	0,67 / 66k	3/6	33,1k (82×)

Fuera de ventana (misma convención; `direct` infactible salvo en las tareas chicas):

paradigma	C2	C3	C4	C5	flips	costo mediana (rango)
direct	INFACTIBLE	INFACTIBLE	INFACTIBLE	1,00 / 23k*	0/8	22,9k (1×)
react	0,97 [†] / 61k	1,00 / 14k	1,00 / 18k	1,00 [†] / 40k	0/7	14,6k (40×)
map_reduce	0,95 / 107k	0,00 / 276k	1,00 / 29k	0,00 / 23k	1/8	28,7k (15×)
plan_execute	0,62 / 88k	0,33 / 18k	0,33 / 80k	0,33 / 94k	4/8	34,4k (52×)
reflection	1,00 [†] / 106k	1,00 / 50k	1,00 / 17k	1,00 [†] / 89k	0/7	30,0k (47×)
dag_strategy	1,00 [†] / 50k	0,67 / 86k	1,00 / 14k	1,00 [†] / 26k	1/7	21,0k (59×)

* sólo en tareas cuya propia evidencia entra: la poda es por tarea, no por corpus.

Tres hallazgos que el primer estudio no podía ver:

El ranking se invierte con el régimen, según la predicción y más allá de ella. En ventana, los lectores dominan en calidad, costo y estabilidad a la vez. Fuera de ventana no existen, y los paradigmas que el primer estudio rankeó últimos — `react`, `reflection`, `dag` — ocupan la cima de la tabla. `plan_execute` es la excepción en ambos regímenes: sin región ganadora, y fuera de ventana es lo menos estable que se midió (4 de 8 celdas dadas vuelta).

El cero de `map_reduce` en las acopladas es estructural, y ahora está medido a dos escalas (P3, confirmada): 0,00 en ventana y 0,00 fuera, deterministamente — sus fallos ni siquiera se dan vuelta. §8.3 carga la mitad de costo de esto.

La estabilidad vive donde el retrieval no decide. En ventana, los paradigmas con herramientas dieron vuelta 3 de 6 celdas cada uno mientras los lectores ninguna. Fuera de ventana, `react` no dio vuelta ninguna de 7 — sin el competidor leer-todo, su búsqueda tiene un trabajo que sí puede terminar — y la inestabilidad migró a `plan_execute`. La varianza no es propiedad de un paradigma; es propiedad de *a quién se le pide decidir cuándo parar*.

7.5 La topología elaborada contra el fallback general

La pregunta que este estudio existe para responder en lugar del reporte de campo retirado: ¿la topología más elaborada le gana al fallback general? La respuesta medida es que la pregunta tiene forma de régimen — y que donde más importa, el diferenciador no es la topología.

En ventana, `dag_strategy` es la peor compra medida: el rango de costo más ancho (82×, de 3,1k a 251k sobre sets idénticos), un 0,33 en las acopladas que `direct` resuelve por 11k, y 3 de 6 celdas dándose vuelta entre réplicas. Fuera de ventana se transforma: utilidad perfecta en C2, C4 y C5 a costos que rivalizan o superan los de `react` †.

Excepto en la cadena acoplada profunda, donde produjo la peor fila individual del estudio: **395.960 tokens en 35 iteraciones para un cero**, en una tarea que `react` resuelve por 10–14k con utilidad 1,0 en las tres réplicas. Esta corrida usó la superficie **basic** — sin señales contables — y la traza muestra el mecanismo de §8.2 a escala: el verificador sigue encontrando la respuesta incompleta, el replan sigue ensanchándose, y nada en el entorno dice *basta*. §7.3 midió que las señales cortan exactamente ese bucle 3,05×. Juntos, los dos resultados dicen algo más filoso que "el DAG pierde": **la topología elaborada es**

viable fuera de ventana sólo bajo contabilidad impuesta desde afuera — lo que faltaba no era inteligencia del bucle.

7.6 Dos paradigmas entran por predicción registrada — uno sobrevive

Contra las dos raíces de falla de §8 que ningún paradigma existente ataca barato, agregamos dos paradigmas con predicciones registradas antes de correr (P6–P7, README del harness) y los tamizamos sobre las tareas discriminantes fuera de ventana.

rewoo — todas las tool calls planificadas en una pasada con placeholders de dataflow explícito, ejecutadas sin el modelo en el loop, una llamada de resolución; dos llamadas LLM en total, sin reenvío de historial [ReWOO, arXiv:2305.18323]. Las dos predicciones registradas se cumplieron, la primera más allá de su cota declarada: calidad idéntica a **react** en cobertura independiente (0,88 en C2, 1,00 en C4) al **3–8% del costo de react** en las mismas celdas (la predicción decía $\leq 50\%$), y utilidad 0,00 en las celdas acopladas y de horizonte desconocido — un plan que no puede observar no puede descubrir el hop que depende de un resultado previo. Sus utilidades fueron idénticas entre réplicas en las cuatro tareas. Un especialista de manual del Teorema 1: G grande dentro de una región de borde nítido, fallo total afuera.

gist_reader — una tabla determinística de gists por unidad en un prompt, después lecturas completas dirigidas y batcheadas. **Su predicción principal fue falsificada:** se predijo $u \geq 0,75$ en tres celdas y se alcanzó en una (C5: 1,00 a 8k donde **react** paga 20–103k); en cobertura masiva y agregación exacta los gists de 312 caracteres no cargan el dato (0,29 en C2, 0,00 en C4) — precisamente el mecanismo de summary-failure que la predicción secundaria nombraba. Su cota de cardinalidad también se comportó según lo registrado: la tabla de gists queda infactible por aritmética en tareas de 400 unidades, registrado gratis. El paradigma queda medido y no promovido. Lo reportamos con el mismo largo que el éxito a propósito: la disciplina de predicciones registradas sólo vale la pena si una falsificación cuesta un párrafo y no una retractación.

7.7 La superficie decide más que la topología, medido dos veces

Dos manipulaciones de la superficie de herramientas, sobre los mismos brazos y las mismas tareas, con la misma forma de resultado.

Ofrecer estado compartido. Un blackboard es la afordancia obvia de coordinación para topologías multi-agente, y la única capacidad de este registro que no es recuperación: **post** escribe un hallazgo que sobrevive la compactación, **board** lee lo que todos posteron. Ofrecido sobre doce paradigmas y 46 celdas ejecutadas, de las **125 llamadas a herramientas que el modelo hizo**, **post se llamó una vez y board ninguna** — 0,8%. Mientras tanto el *mismo objeto*, escrito por el código y renderizado en el prompt de cada sub-agente, pertenece al mejor paradigma fijo sobre el mundo held-out.

Ofrecer una forma de leer todo de una vez. Exponerle a un brazo una herramienta de lectura completa, sobre las mismas tareas, lo volvió **1,57× más barato en tokens a utilidad idéntica** — **+0,000** sobre 63 celdas pareadas. La herramienta se invocó en **3 de 63 celdas**, y las unidades leídas *bajaron*. El ahorro vino de otro lado: los caracteres releídos cayeron de 2.836.465 a 322.094, **8,8×**. El brazo leyó menos y se repitió mucho menos, con la misma calidad.

En los dos casos el efecto está en la oferta, no en el uso. Lo que se le ofrece a un agente cambia lo que hace, en buena medida con independencia de lo que llama — y un benchmark que puntúe herramientas por tasa de invocación mide la variable equivocada.

Y la traza por llamada muestra a dónde va la plata, cosa que ninguna fila agregada podía: el primer turno de un bucle cuesta 607 tokens de prompt y el octavo cuesta 67.233, **110x**. Sobre el brazo entero, **el 99% del gasto de entrada es la conversación mandada otra vez**. El costo en tokens crece con el cuadrado de las vueltas mientras la cobertura crece linealmente — una propiedad del transporte, no del modelo. El caché del proveedor absorbe cerca de la mitad de esa repetición a una décima parte del precio, así que el mismo hecho se lee como **1,57x en tokens y 1,36x en dólares**; un resultado de costo sin su unidad no es reportable.

7.8 Qué necesita un engine de selección para funcionar

La literatura reporta el premio de la selección de paradigma como una brecha y lo persigue con mejores selectores. El registro completo permite enunciar, en cambio, **las condiciones que un engine de selección tiene que cumplir antes de que valga la pena construir un selector** — y dónde está el nuestro contra cada una.

Los brazos tienen que separarse más de lo que la medición se separa a sí misma

Descomponiendo la varianza de la utilidad sobre 1.284 filas medidas:

fuelle	varianza	del total
total	0,2469	
entre tareas	0,1153	47%
entre paradigmas	0,0311	13%
entre réplicas de una celda	0,0311	13%
interacción tarea × paradigma (residual)	0,0694	27%

La cuarta fila es la que faltaba, y la agregamos porque su ausencia hacía que la tabla no cerrara: $47 + 13 + 13 = 73\%$, no 100%. El residual es la interacción, y es el término que §7.11 mide después con el método completo sobre otro panel (48% ahí, con n distinta y descontando el ruido). Que sea grande **no contradice** lo que sigue: §7.11.3 muestra que esa interacción, restringida a los brazos que competirían, cae a $0,0046$.

0,0311 contra 0,0311 — iguales a la cuarta decimal. Un router elige paradigma, así que sólo puede competir por la porción que el paradigma explica; la de la tarea no la mueve ninguna política y la de réplica es ruido por construcción. Acá la señal sobre la que un router elige tiene exactamente el tamaño del ruido contra el que se la mide.

Es chequeable antes de gastar nada, y barato: la descomposición necesita réplicas y varios brazos, no un producto cruzado completo. Debería ser la primera pregunta que se le hace a un corpus, y nosotros la hicimos última.

El eje sobre el que segmenta tiene que ser recuperable del request

Un engine de selección parte los requests y aprende por partición, así que la partición tiene que ser computable al momento de decidir. La nuestra no lo es, y la demostración es un contraejemplo y no una correlación. Dos celdas:

	C5_unknown_horizon	C8_currency
todos los campos computables del request	idénticos	idénticos
región asignada	<code>*/no_oracle/loose/chain</code>	la misma
correlación de cobertura con utilidad	+0,331	-0,373

C5 pregunta qué individuo tiene información de ciudad *contradictoria* **entre** las unidades — la contradicción sólo se ve después de leerlas todas. C8 pide el domicilio **actualmente** en archivo — la respuesta es el más reciente entre registros que compiten, así que leer más aporta más candidatos viejos. **Lo que decide es una propiedad de lo que la pregunta significa, y ningún mapa de features sobre el request declarado las separa.**

Y es también por qué el premio es chico y está concentrado: de 43 tareas con los nueve brazos generales, **35 no tienen brecha alguna** —el mejor brazo fijo ya es el oráculo— y **5 tienen un único mejor brazo**, tres de ellas ganadas por un brazo que promedia 0,440 en el corpus y 0,917 adentro de esa celda. **La selección no se gana eligiendo el brazo que suele ser bueno; se gana sabiendo cuándo el que suele ser malo es el correcto**, y el margen de error de un router es en consecuencia minúsculo.

El objetivo tiene que contener el premio

Clasificando las tareas por qué clase de decisión presentan en realidad:

clase	tareas	brecha de calidad	ratio de costo
los brazos difieren en calidad	16 (35%)	0,568	11,2×
la mayoría empata	21 (46%)	0,181	57,0×
nadie la resuelve	9 (20%)	0,000	51,3×

En el 66% de las tareas no hay nada que elegir en calidad, y brazos que devuelven la misma respuesta difieren 50× en costo. El engine ordena sólo por utilidad media; el costo está medido, guardado, y nunca se lee al elegir. Así que el premio que existe en este corpus es uno que el objetivo no puede expresar.

Y plegar el costo dentro del objetivo no lo arregla, cosa que probamos antes de proponerla. Ordenar por $u - \lambda \cdot \text{costo}$ *baja* la señal entre brazos para todo λ moderado, y sólo se recupera en $\lambda = 1$, donde ya no se rutea por calidad en absoluto. El motivo es medible: **el costo es más ruidoso entre réplicas que la calidad** — varía más de 2× entre réplicas de la misma celda en 99 de 428 celdas, contra una dispersión media de utilidad de 0,141. La misma varianza que hace que el costo valga la pena optimizar es la que lo hace difícil de aprender.

Y cuando esas condiciones fallan, el engine tiene que negarse

Ajustado sobre 428 episodios en 8 regiones, **dos regiones tienen dos o más brazos por encima del piso de evidencia** —las primeras del proyecto— con márgenes de **0,0417 y 0,0381** contra un piso de ruido por celda de **0,1407**. Al umbral configurado la política no opina en ninguna.

Ése es el comportamiento correcto y vale enunciarlo como resultado positivo. No hay umbral de abstención que ayude: cualquier valor por debajo de 0,14 hace que la política decida sobre diferencias más chicas que la dispersión entre dos corridas de la misma celda. **El engine no está fallando en decidir — le están dando bins donde la respuesta correcta es «da igual», y lo dice en vez de adivinar.**

El engine funciona; el corpus y el vocabulario no sostienen lo que se le está pidiendo decidir.

Son afirmaciones separables, y separarlas es para lo que sirven las cuatro condiciones de arriba. Un resultado negativo de selección que no las reporte no puede distinguir «la selección no paga» de «este montaje no lo puede ver».

7.9 La corrida homogénea completa: los doce paradigmas sobre las 78 tareas

Las secciones anteriores midieron sobre 41 de las 78 tareas del corpus, y lo que faltaba no era aleatorio: **21 de las 32 sin medir eran del ancho mayor**, 60 unidades y ~483.000 tokens de material por tarea. Ése es exactamente el régimen donde las topologías deberían separarse —donde leer todo es imposible y la cobertura exhaustiva no se puede pagar— así que toda conclusión anterior estaba acotada al régimen estrecho sin decirlo.

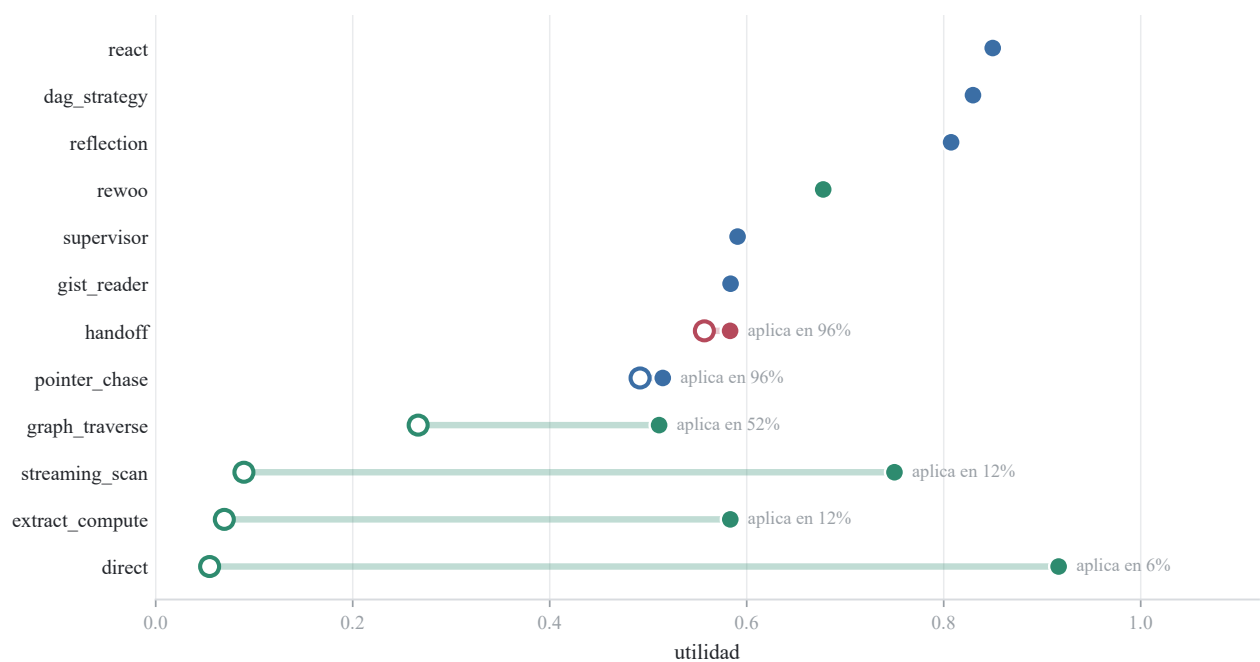
Esta sección cierra el corpus. Doce paradigmas × 78 tareas × 3 réplicas, un modelo, mismas condiciones. Sin errores de infraestructura.

7.9.1 Un brazo se mide dos veces, y las dos son distintas

brazo	aplica	u donde aplica	u × cobertura	tokens/celda
react	100%	0,850	0,850	108.137
dag_strategy	100%	0,830	0,830	105.293
reflection	100%	0,803	0,803	133.574
rewoo	100%	0,669	0,669	10.840
gist_reader	100%	0,584	0,584	23.075
supervisor	100%	0,581	0,581	64.079
handoff	96%	0,583	0,557	131.310
pointer_chase	96%	0,515	0,492	9.787
graph_traverse	52%	0,511	0,266	21.356
streaming_scan	12%	0,750	0,090	26.380
extract_compute	12%	0,583	0,070	26.184
direct	6%	0,917	0,055	22.822

Un paradigma tiene dos números, y colapsarlos esconde el caso extremo

relleno: utilidad donde su mecanismo corre · hueco: multiplicada por su cobertura



Bueno donde aplica, contra lo que aporta sobre el corpus

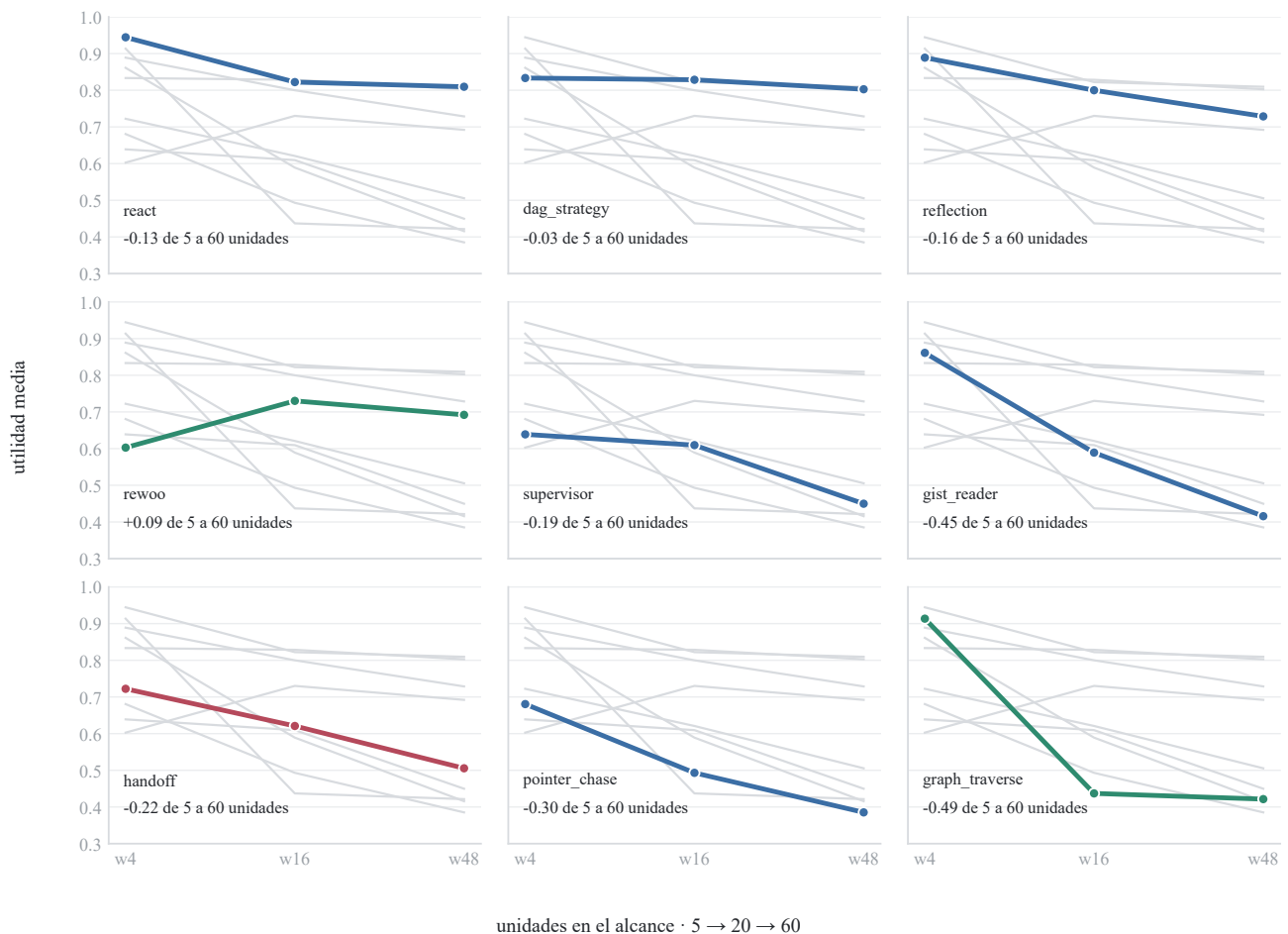
Un paradigma tiene dos números y colapsarlos esconde el caso que importa. `direct` es el mejor del plantel donde su mecanismo corre —0,917— y su mecanismo corre en el **6%** de las celdas —**12 filas de 201**, y ese número va junto al porcentaje porque «el mejor del plantel» sobre doce filas es una afirmación de otra clase que sobre doscientas—, porque la aritmética de factibilidad lo poda en cuanto el material no entra. Sobre el corpus aporta 0,055. Reportar un solo número obliga a elegir cuál de las dos afirmaciones falsear.

La cobertura no es una propiedad del brazo sino de la **intersección** entre su mecanismo y la distribución de tareas, y por eso se decide antes de gastar: `direct`, `streaming_scan` y `extract_compute` no fallan en el 88–94% restante — **no corren**.

7.9.2 La degradación con el ancho separa lo que la utilidad media junta

Casi todos se degradan cuando el material crece. `'rewoo'` no.

un panel por brazo; los demás, en gris de fondo



Cómo se degrada cada brazo cuando el material crece

El eje son los tres anchos declarados —**5, 20 y 60 unidades**—. Las tareas sin sufijo de ancho quedan **fuera**: agrupan celdas de 1, 8, 9 y 60 unidades, así que no son el extremo angosto de nada y meterlas convertía el eje en algo que no está ordenado.

brazo	w4 (5 u.)	w16 (20 u.)	w48 (60 u.)	Δ
react	0,94	0,82	0,81	-0,13
dag_strategy	0,83	0,83	0,80	-0,03
reflection	0,89	0,80	0,71	-0,18
rewoo	0,60	0,73	0,66	+0,06
handoff	0,72	0,62	0,51	-0,22
supervisor	0,64	0,59	0,43	-0,20
gist_reader	0,86	0,59	0,42	-0,45
pointer_chase	0,68	0,49	0,39	-0,30
graph_traverse	0,91	0,44	0,42	-0,49

Todos se degradan menos uno. `graph_traverse` pierde 0,49 y `gist_reader` 0,45 al pasar de 5 a 60 unidades — el gist de 180 caracteres y el índice de entidades dejan de discriminar cuando hay sesenta candidatos. **rewoo es la única excepción**, y sube: +0,06. No es casualidad — es el único brazo cuyo costo no es función del alcance.

Un promedio sobre anchos habría llamado «mejor» a `gist_reader` que a `rewoo` por 0,584 contra 0,669, y habría escondido que uno se desploma exactamente donde el otro se sostiene.

7.9.3 Tres clases de costo, y no son las del catálogo

Ajustando $\log(\text{costo})$ contra $\log(\text{alcance})$ por brazo, y preguntando por separado qué explica mejor el costo —el alcance o la cantidad de vueltas— aparece una taxonomía que **corta transversal** a la de control de flujo:

clase	brazos	qué la define
alcance	<code>handoff</code>	$R^2 = 0,77$ contra el alcance, exponente 0,73. El costo lo fija cuánto material arrastra cada llamada
vuelatas	<code>react</code> , <code>reflection</code> , <code>dag_strategy</code> , <code>supervisor</code> , <code>gist_reader</code> , <code>pointer_chase</code>	el costo lo fija cuántas veces itera, y eso es endógeno : gasta hasta que algo lo detiene
estructural	<code>rewoo</code> , <code>direct</code> , <code>graph_traverse</code> , <code>extract_compute</code> , <code>streaming_scan</code>	ni una ni la otra: el costo está fijado por la forma del patrón

El dato que obliga a separar estas clases de los techos declarados: **handoff tiene un techo de 12 llamadas y gasta 131.310 tokens; dag_strategy tiene uno de 160 y gasta 105.293**. Casi lo mismo, con un factor 13 de diferencia en el techo.

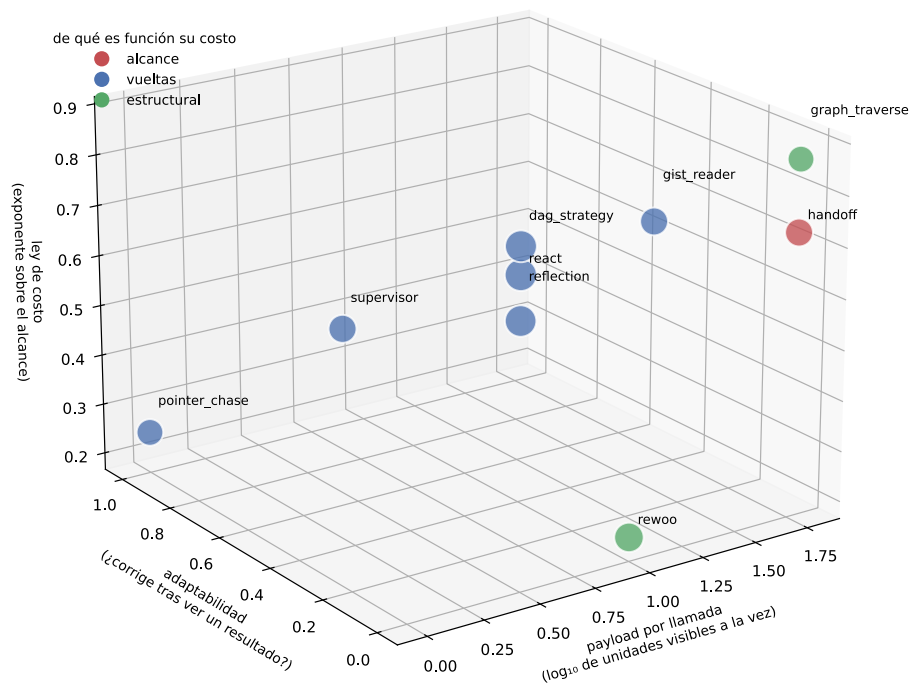
Contar llamadas para acotar esfuerzo es contar envases para acotar peso.

Y una consecuencia operativa: la clase **vuelatas** es la única sobre la que una regla de parada puede actuar. Medido en el mismo registro, el 46,5% de las búsquedas de `react` no traen ninguna unidad nueva, con

rachas de hasta 14.

7.9.4 El espacio de capacidades

El espacio de capacidades: cada brazo, un punto
el tamaño es su utilidad; el color, de qué es función su costo



El espacio de capacidades

La taxonomía de control de flujo —«plan-ejecuta», «supervisor», «cadena»— no predice rendimiento. Lo que sí predice es qué **capacidades** le da cada topología al modelo, y son tres:

eje	qué es	de dónde sale
payload por llamada	cuántas unidades ve el modelo de una vez	del código
adaptabilidad	¿puede corregir el plan tras ver un resultado?	del código
ley de costo	de qué es función su costo	medida

Las dos primeras se leen del código, y por eso **ubican a un brazo que todavía no se corrió** — cosa que una tabla de resultados no puede hacer.

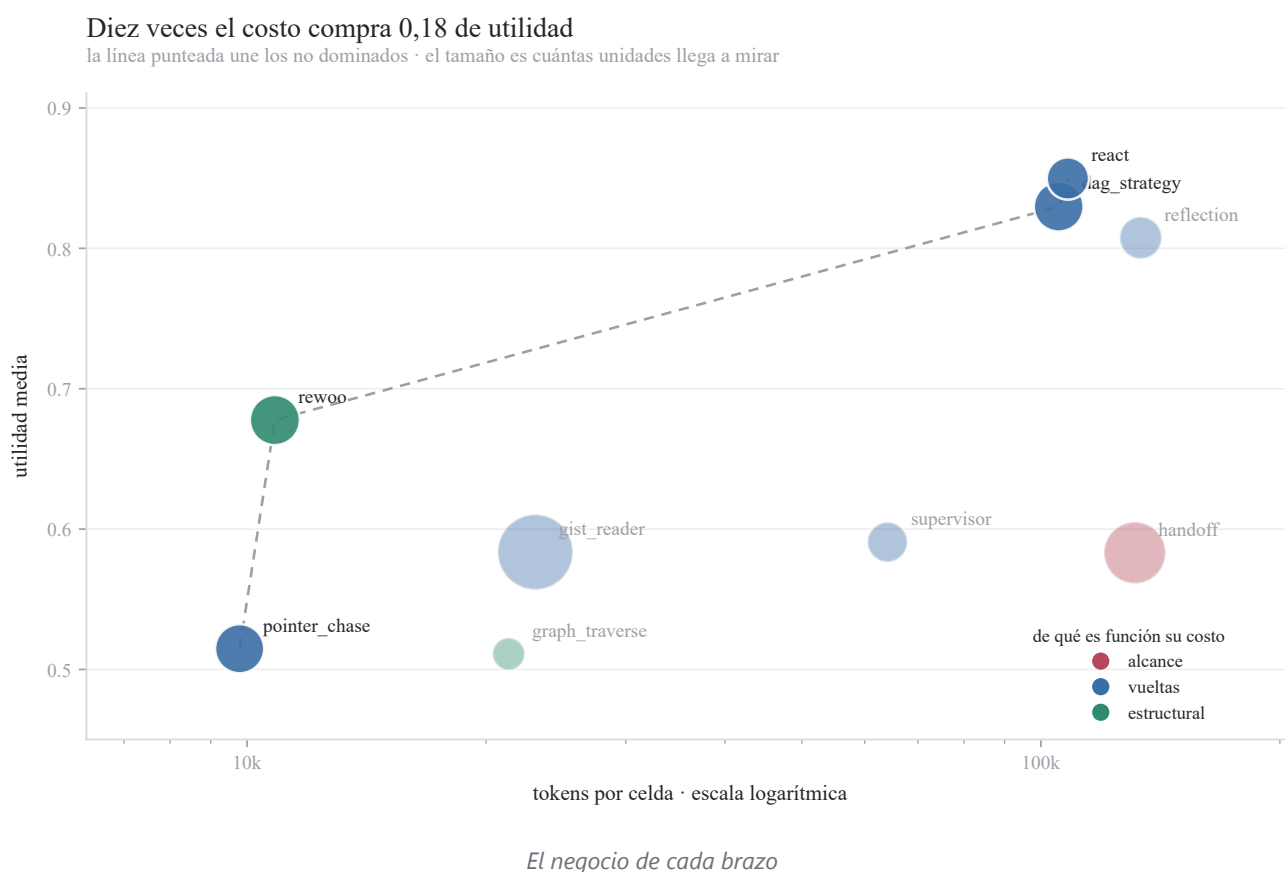
El caso que las valida es la pregunta de contradicción, donde la respuesta es una relación entre dos unidades y ninguna unidad la contiene:

capacidades	utilidad
payload $\geq$ 2 unidades y adaptabilidad	0,71 – 0,91
sólo payload	0,13
ninguna	0,00 – 0,40

handoff **lee las dos unidades relevantes y saca 0,067**, porque cada sub-agente ve su mitad y ninguna llamada tiene el par: la capacidad no es «leerlas» sino «tenerlas juntas». Y rewoo saca 0,133 aunque *puede* tenerlas juntas, porque le falta la otra — poder elegir **cuáles** dos exige ver un resultado antes de pedir el siguiente.

En la figura, react, dag_strategy y reflection caen prácticamente en el mismo punto. Eso no es un defecto del dibujo: **son el mismo brazo para decidir**, y por eso sus utilidades quedan dentro de 0,05 entre sí.

7.9.5 Dónde está el margen



En el ancho mayor, react saca 0,81 a 108.137 tokens por celda y rewoo 0,66 a 10.840: **+0,15 de utilidad por un factor 10 de costo**. Y el costo es casi enteramente de entrada —de **98,6% a 100,0%** según el brazo, con la salida entre 0,0% y 1,4%—, lo cual dice que **los paradigmas no se diferencian en lo que generan sino en lo que arrastran al prompt**. Es la misma afirmación que la ley de costo, medida por otro lado.

Un brazo queda afuera de ese rango y hay que decirlo: en `graph_traverse` la suma de entrada y salida da 101,9% del costo registrado. Entrada y salida son las dos particiones del mismo total, así que por encima de 100% no hay una tercera categoría: **es un defecto de contabilidad de ese brazo**, no una propiedad medida. Se excluye del rango y queda anotado como deuda, no como hallazgo.

7.10 Dónde falla, y qué lo arregla

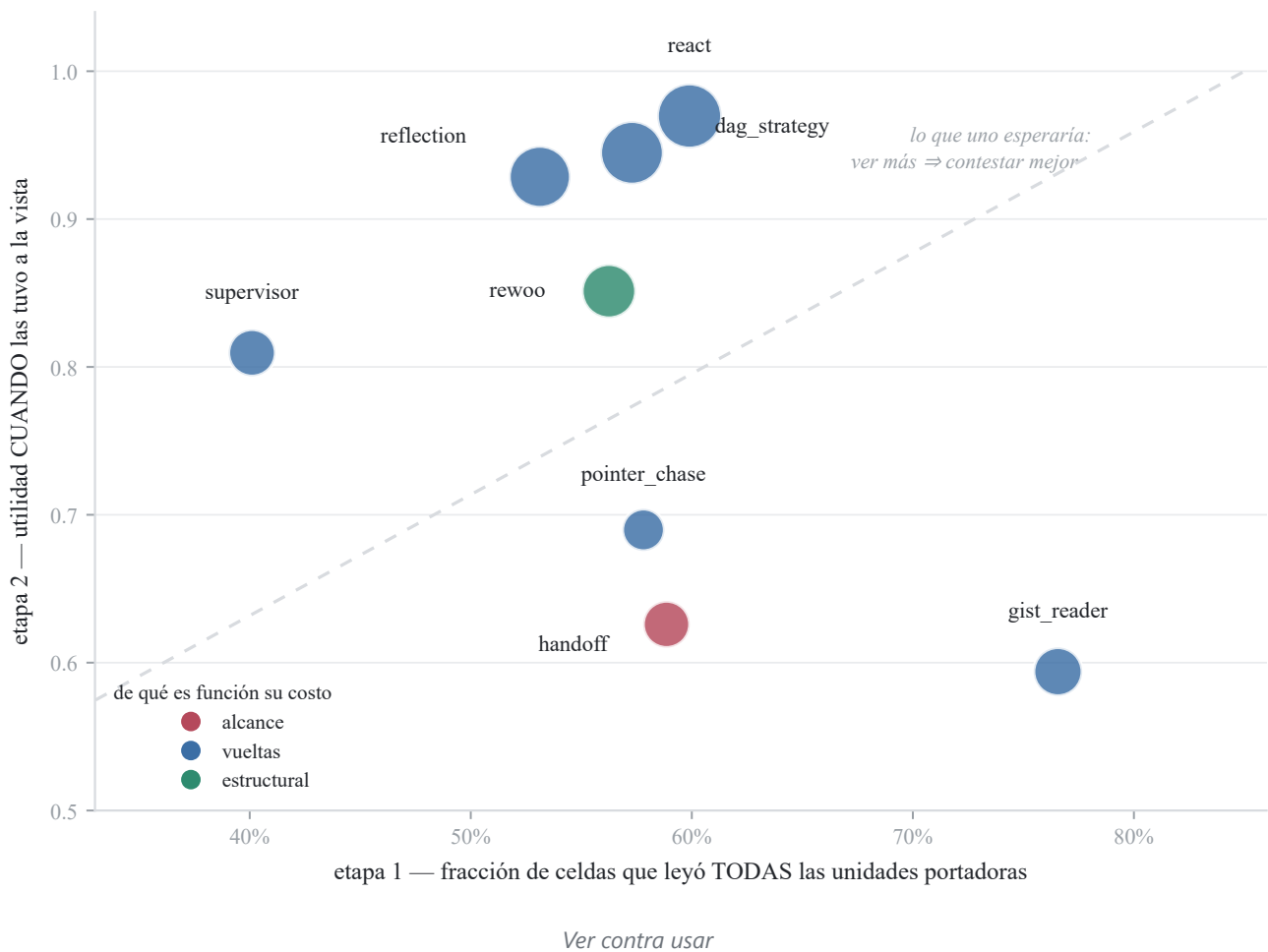
Las secciones anteriores comparan brazos. Ésta abre uno: **por qué gana el más simple, qué varianza esconde el promedio, y qué pasa cuando una decisión de control se le saca al modelo**. Las tres preguntas se contestan sobre el mismo registro, sin gastar un token más.

POR QUÉ u **CAMBIA DE VALOR ENTRE SECCIONES, y no es un error.** `react` figura con 0,850 en §7.9.1, 0,843 en §7.10.1 y 0,875 en §7.10.2. Son **tres paneles distintos**, y compararlos sin decirlo sería el defecto que `bench/pane1.py` existe para impedir: | sección | panel | por qué ése | |---|---|---| | §7.9.1 | 78 tareas × 12 brazos, cada brazo sobre las celdas donde CORRIÓ | mide cobertura y aporte, que exigen incluir a los brazos podados | | §7.10.1 | 64 × 8 (rectángulo completo) | el embudo compara etapas entre brazos: exige que todos hayan corrido las mismas tareas | | §7.10.2 y §7.11 | 59 × 8 (rectángulo con ≥3 réplicas) | $pass^3$ y la descomposición necesitan las tres réplicas de cada celda | **La regla del rectángulo está en un solo lugar** (`bench/pane1.py`) y devuelve también lo descartado, porque un panel que se achica sin decir cuánto miente por omisión. Toda u reportada usa $\lambda = 0$ —calidad pura, con el costo en su propia columna— salvo donde se indique lo contrario.

7.10.1 El embudo: `react` no gana buscando

`react` no gana buscando: gana no destruyendo lo que encontró

área del punto = utilidad sobre el corpus · 64 tareas x 8 brazos (82% de las 78 medidas)



La utilidad de una celda es el producto de dos cosas que fallan por razones distintas:

$$u \approx P(\text{vio TODAS las unidades portadoras}) \times P(\text{contestó bien} \mid \text{las vio})$$

brazo	u	vio	**u \	vio**	u \	no vio	leídas
react	0,843	60%	0,970	0,708	7,8		
dag_strategy	0,822	57%	0,945	0,720	11,6		
gist_reader	0,611	77%	0,594	0,667	28,8		
supervisor	0,577	40%	0,778	0,479	6,8		

react no ve más que nadie — 60%, y **gist_reader** ve el 77%. Toda su ventaja está en la segunda etapa: con el material a la vista da 0,970, y **gist_reader** da 0,594, *peor que cuando no lo vio todo*. Pareado por tarea, Δvio es chico o negativo y $\Delta(u \mid \text{vio})$ se lleva la brecha entera.

El mecanismo: cada representación intermedia más chica que el material es una pérdida que no se recupera aguas abajo. Un gist de 180 caracteres, una ventana recortada para el sub-agente, un índice de entidades — los tres tiran información *antes* de saber cuál hacía falta. **react** no tiene ninguna.

La simplicidad no es acá una virtud estética: es la **ausencia de un canal con pérdida**, y eso se mide.

7.10.2 `pass^k` : el promedio esconde la mitad que importa

`pass^k` **NO** es `pass@k` , y significa casi lo opuesto. En la literatura de código `pass@k` mide «al menos un acierto en k intentos» y **crece** con k . `pass^k` mide «los k intentos acertaron todos» y **decrece** con k . El nombre viene de `tau2-bench` , que lo introdujo para agentes multi-turno por esta misma razón; lo conservamos por consistencia con esa literatura y lo señalamos acá porque el parecido tipográfico invita a leer la tabla al revés.

Todo lo anterior es `pass@1` —el promedio sobre réplicas— y responde «cuánto acierta». `pass^k` responde «**se puede contar con que acierte**», que para un sistema que promete «misma base de creencias $\Rightarrow$ misma decisión» es la mitad que importa.

brazo	<code>pass@1</code>	<code>pass^3</code>	caída	celdas inestables
<code>react</code>	0,875	0,797	−0,078	17%
<code>dag_strategy</code>	0,874	0,763	−0,112	20%
<code>rewoo</code>	0,718	0,576	−0,142	27%
<code>supervisor</code>	0,637	0,441	−0,196	34%

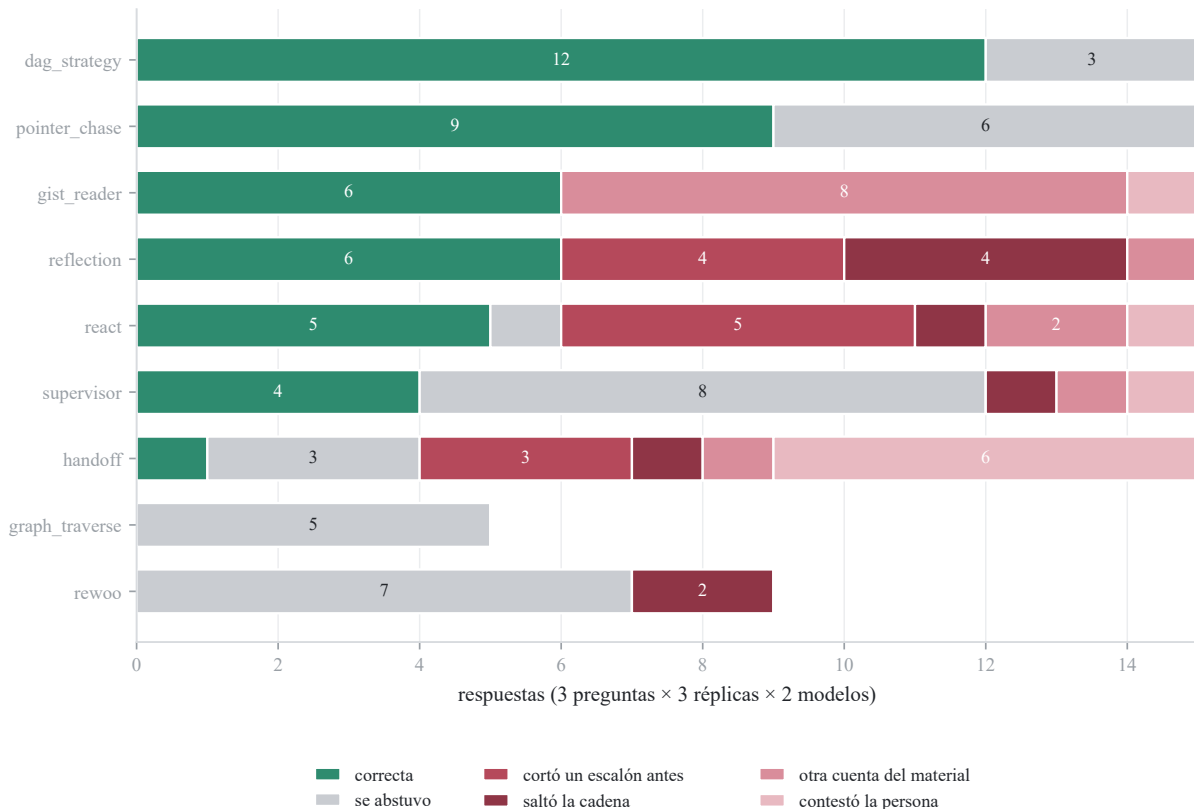
Entre el 17% y el 34% de las celdas cambian de resultado entre réplicas, con $t=0$, semilla fija y la misma huella. Eso **corrige una afirmación previa nuestra**: el determinismo verificado con una llamada por modelo es cierto *por llamada* y falso *por trayectoria* — un bucle de herramientas amplifica cualquier desvío, porque una elección distinta en el paso uno cambia todo lo que sigue.

Ningún piso de ruido *entre brazos* muestra esta varianza: vive **dentro** de una celda.

7.10.3 El caso C3: el modo de falla no era el que parecía

C3: 'dag_strategy' no encadena mejor — es el único que se calla

el banco puntúa «se abstuvo» y «contestó mal» los dos con 0,000, y esa es la diferencia que el producto vende



Modos de falla de C3

C3_coupled_chain pide subir N escalones de una línea de reporte sobre 60 unidades y reportar un dato del último. La cadena está minada a propósito: **cada unidad del camino lleva un dato del mismo tipo pegado al nombre que la ancla**, el enlace va por anáfora («The above-named», «That person»), y el destino del salto viene abreviado (A. Vallejos apunta a Agustina Vallejos).

Clasificar las respuestas por modo, y no por puntaje, invierte el diagnóstico. El modo dominante no es saltarse la cadena (2 casos) sino **cortarla un escalón antes** (7): los brazos devuelven el dato de un intermedio, que está a la vista y es indistinguible del correcto. Y dag_strategy, que gana la celda, **no encadena mejor**: hace 12 correctas, 3 abstenciones y **cero respuestas equivocadas**. Los demás contestan igual.

El banco puntúa «se abstuvo» y «contestó mal» los dos con 0,000. Es correcto para medir utilidad y **ciego justo en el eje que la capa de decisión existe para gobernar**.

7.10.4 Sustituir una decisión del modelo por un sensor determinista

Sobre ese diagnóstico se corrigió pointer_chase —el brazo cuyo mecanismo es seguir cadenas y que sacaba 0,33 en C3—. Cuatro correcciones, **todas de flujo de control o de tipado, ninguna de fraseo**:

1. ****Regla de creencias**: una entidad nombrada se busca con el índice léxico, no con el

híbrido.** Un vector denso codifica *de qué habla* un texto, y sesenta documentos con la misma plantilla hablan de lo mismo; el nombre propio es justo la parte que **no** es semántica, y fusionarle la rama densa le

mete ruido a la única señal que discrimina. Medido: el híbrido devuelve la unidad equivocada para Ramiro Herrera y deja a M. Arrieta fuera del top-5; el léxico las pone primera y tercera.

1. **La salida del sensor se tipa antes de usarse.** El modelo emitía

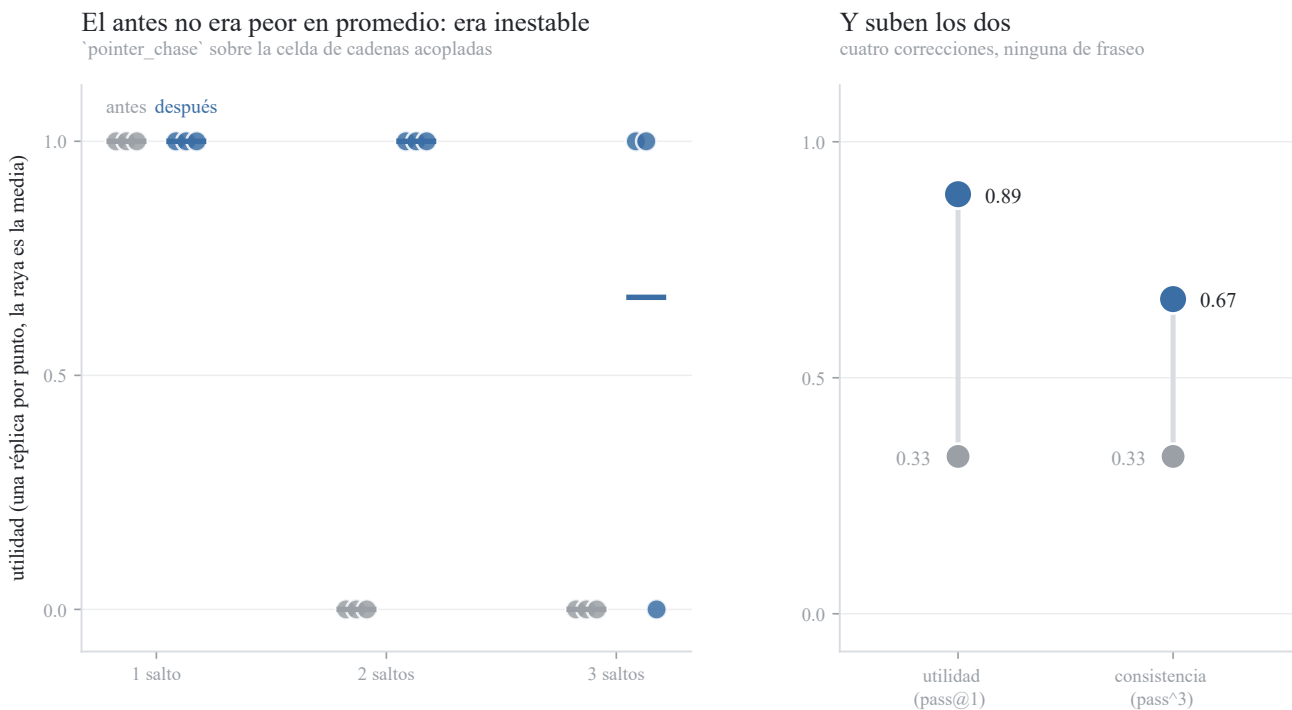
'M. Arrieta settlement account', y esa cola arrastraba la consulta a clasificar como prosa: **la regla era correcta y la entrada estaba sucia.**

1. **Un salto a una unidad que no nombra a quien se persigue no es un salto**, y entre

candidatos empatados gana el que la nombra **antes** — un documento que trata *sobre* una entidad la nombra antes que uno que la referencia de paso. Se resuelve con predicados que devuelven un booleano o una posición, nunca texto: no cuestan un token.

1. **El ancla es el salto cero y lo resuelve el código.** Era una llamada al modelo, y ahí

quedaba la última decisión de flujo en manos del sensor.



El sensor determinista

El panel izquierdo dibuja **cada réplica por separado**, y ahí se ve lo que un promedio esconde: el «antes» no era peor en promedio sino **inestable** — la misma pregunta, la misma huella y los mismos resultados de búsqueda daban 1,000 o 0,000 según la réplica. El panel derecho muestra los dos ejes moviéndose juntos, que es la afirmación entera.

Resultado: `pointer_chase` **pasa de 0,33 a 0,89 en C3**, empatando al mejor brazo de la celda. Y la corrección (4) prueba el punto por sí sola: **con la misma huella y los mismos resultados de búsqueda**, la réplica 0 elegía el ancla correcta y recorría la cadena entera ($u=1,000$) mientras las réplicas 1 y 2 elegían otra y sacaban 0,000. Una decisión de flujo en el sensor se lleva puesto el determinismo.

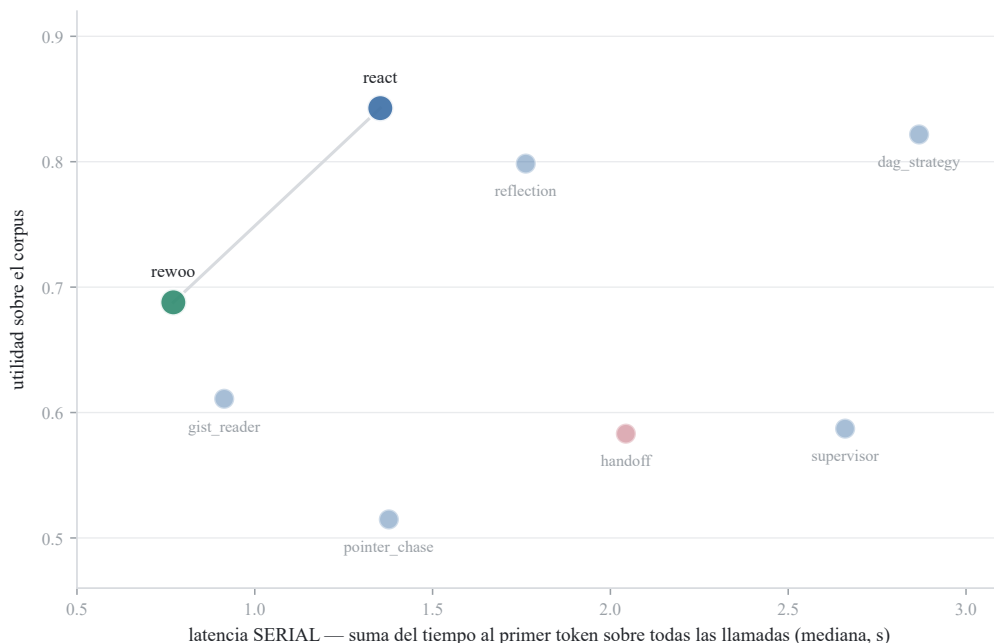
La hipótesis, y es falsable: sustituir una decisión de control del modelo por un sensor determinista sobre una señal del entorno mejora **la utilidad y el determinismo a la vez**. `pass^k` es la métrica que faltaba para medir el segundo efecto, y sin ella la mitad de la mejora era invisible.

Lo que queda declarado y no resuelto: la réplica que todavía falla **se abstiene** en vez de contestar mal, que es el comportamiento buscado y que el banco puntúa igual que un error.

7.10.5 Un eje medido que ninguna decisión mira

El eje nuevo no destraba nada: `'react'` es el mejor Y el más rápido

el frente une a los no dominados · el primer token es igual en todos (250–410 ms); lo que cambia es cuántas llamadas van en serie



Latencia serial

Hay **dos relojes** y confundirlos invalida el número. El tiempo de pared no sirve: el 63-67% de las filas de `react` y `dag_strategy` están por debajo de medio segundo porque son replays del caché en disco — eso mide cuánto tarda el banco en releer, no cuánto tarda el sistema en contestar. Lo que sí sirve viene del proveedor, en el objeto `usage` de cada respuesta, y por eso el caché lo conserva: es la latencia de la llamada que efectivamente se hizo.

brazo	u	primer token	latencia serial	u por segundo
react	0,843	265 ms	1,35 s	0,62
dag_strategy	0,822	410 ms	2,87 s	0,29
rewoo	0,677	304 ms	0,77 s	0,88
supervisor	0,577	360 ms	2,66 s	0,22

El primer token es prácticamente igual en todos —250 a 410 ms, es una llamada al mismo modelo—. Lo que cambia 3,6× es la latencia **serial**: la suma sobre todas las llamadas, o sea la parte que no se acelera con más tokens por segundo porque cada llamada espera a la anterior. Es la ley de costo `turn-driven`

cobrada en tiempo del usuario en vez de en tokens, y es un eje que **ninguna decisión del ruteador mira hoy**.

Y no destraba nada, que es lo que hay que decir: `react` es a la vez el de mayor utilidad y el de menor latencia serial entre los contendientes. El único que compra tiempo es `rewoo` —1,8× más rápido— y cuesta 0,165 de utilidad. Es un intercambio explícito, no un almuerzo gratis: sólo lo compra quien tenga un techo de latencia declarado.

7.10.6 ¿Se compone el no-determinismo con cada decisión? La forma simple es falsa

La sección anterior invita a una conjetura general, y vale la pena escribirla porque **el registro la refuta en su forma ingenua**:

Si un brazo delega d decisiones de control al modelo, y cada una vuelve a salir igual entre réplicas con probabilidad q , entonces $\text{pass}^k \approx \text{pass}@1 \cdot q^d$. Más decisiones en el sensor $\Rightarrow$ menos determinismo, multiplicativamente.

Es contable: d se mide como iteraciones por celda, y `pass^3` ya está. Sobre los ocho brazos del panel:

brazo	decisiones	pass@1	pass^3	q implícita
<code>dag_strategy</code>	8,9	0,822	0,703	0,983
<code>supervisor</code>	8,6	0,587	0,406	0,958
<code>reflection</code>	5,9	0,798	0,688	0,975
<code>react</code>	4,3	0,843	0,734	0,968
<code>pointer_chase</code>	3,8	0,515	0,359	0,909
<code>rewoo</code>	2,0	0,688	0,531	0,879
<code>gist_reader</code>	1,9	0,611	0,516	0,913

La correlación entre número de decisiones y caída de `pass^3` es $r = -0,24$ con $n = 8$ —débil, y con el signo **contrario** al que la conjetura predice: los brazos con más decisiones pierden *menos*. La explicación que el propio dato sugiere es que una decisión no sólo agrega varianza sino también **una oportunidad de corregir**: un brazo adaptativo que dobla mal puede volver, y uno de dos llamadas no. Los dos efectos casi se cancelan en este corpus.

Lo que sí se sostiene, y es más útil que la conjetura original, son dos cosas:

1. **q está acotada lejos de 1 para todos**. El máximo es 0,983 (`dag_strategy`) y el mínimo 0,879 (`rewoo`). Ninguna arquitectura de las medidas recupera la reproducibilidad por decisión, así que **toda trayectoria con decisiones delegadas pierde determinismo**, y la pregunta no es si se pierde sino cuánto.

1. **Importa más CUÁL decisión se saca que CUÁNTAS**. La evidencia acá no es correlacional sino una intervención: sacar **una sola** decisión —el ancla— llevó a `pointer_chase` de réplicas que discrepaban (1,000 / 0,000 / 0,000) a réplicas que coinciden, sin tocar las otras. Ocho puntos de correlación no compiten con eso.

El no-determinismo no se reparte por igual entre las decisiones de una trayectoria. Contar decisiones no predice; identificar **cuál** decide el resultado, sí.

Ésta es una limitación declarada del análisis, no un resultado: con ocho brazos y un corpus, lo correlacional acá no puede decidir casi nada. Lo que la sostiene es la intervención.

7.11 Por qué no hay premio de ruteo, aunque la interacción sea enorme

Éste es el resultado central del corpus, y es negativo con un mecanismo preciso. La forma en que suele argumentarse que rutear conviene —«hay mucha interacción entre tarea y método, así que elegir por tarea tiene que pagar»— **no se sostiene**, y este registro muestra dónde se rompe.

7.11.1 Hay interacción, y es grande

Descomponiendo $u(\text{tarea}, \text{brazo}) = \mu + \alpha(\text{tarea}) + \beta(\text{brazo}) + \gamma(\text{interacción}) + \varepsilon$ sobre un panel de **59 tareas × 8 brazos**, que es el 76% de las 78 medidas. El criterio de recorte es mecánico y está en un solo lugar del código, no se elige por sección: **primero** las tareas que tienen al menos 7 brazos medidos —lo que impide que una tarea corrida por un experimento parcial redefina el universo—, **después** los brazos que cubren $\geq 95\%$ de éstas, y **al final** las tareas donde están todos. Los cuatro brazos que quedan afuera son los que la factibilidad poda en casi todas las celdas (`direct` , `streaming_scan` , `extract_compute`) más `graph_traverse` , que se declara infactible en el 56-59% de las celdas anchas.

Se declara acá porque ésta es la sección que más insiste en corregir por selección, y un recorte sin criterio sería exactamente lo que audita en otros.

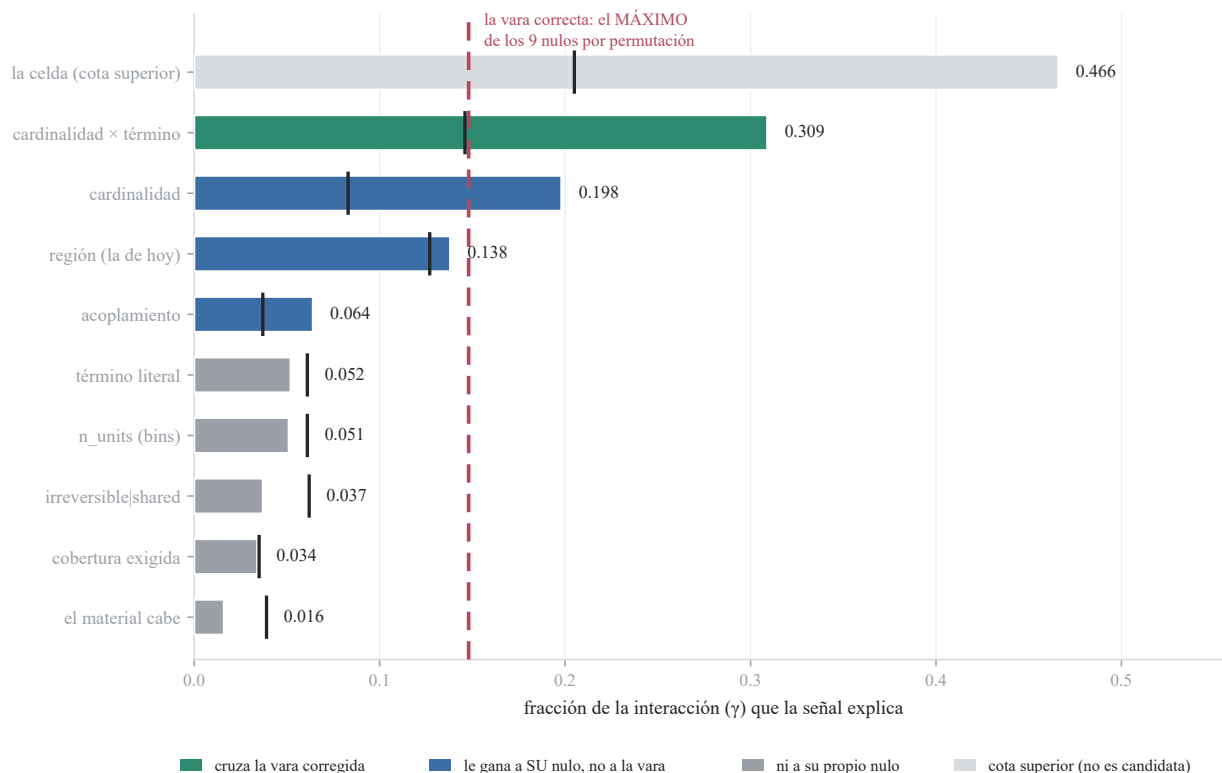
componente	varianza	%
α — dificultad de la tarea	0,0632	41%
β — calidad del brazo	0,0160	10%
γ — interacción	0,0736	48%
ε — ruido entre réplicas	0,0351	

y descontado el ruido da **0,0619**, con **señal/ruido 5,30**. Bajo el argumento habitual, acá habría que rutear.

7.11.2 Y hay una señal que la explica, que sobrevive la corrección por selección

Una sola señal le gana al azar cuando se corrige por selección

la raya negra sobre cada barra es su propio nulo (p95)



Qué señal explica la interacción

La figura ordena diez señales por cuánto de γ explican, con **su propio nulo por permutación** dibujado como una raya negra sobre cada barra. Varias la superan — y esa comparación es exactamente la falacia que el nulo existía para evitar, porque **se probaron nueve candidatas y se eligió la mejor**. La vara correcta es la línea punteada: el **máximo de los nueve nulos en cada permutación**.

Sólo `cardinalidad  $\times$  término literal` la cruza, con 0,309 y p corregido $< 0,001$, y captura **dos tercios del techo** que marca `la celda` — que está en el gráfico como **cota superior**, no como candidata: es la etiqueta de diseño del corpus, no se conoce al decidir, y ninguna señal real puede superarla.

7.11.3 Y el premio neto es negativo

`var( $\gamma$ )` **grande $\neq$ premio de ruteo grande**. El premio es $E[\max_p u] - \max_p E[u]$, y γ puede ser enorme porque los brazos **malos** son malos en lugares distintos. Esa estructura es real, es predecible, y **no vale nada**: nadie va a elegir el brazo que pierde por poco en vez del que pierde por mucho.

Lo único cobrable es la interacción **entre los brazos que competirían**. `gist_reader` aporta el 20% de `var( $\gamma$ )` y `rewoo` el 15%; los tres punteros, 6-7% cada uno. Restringido a esos tres —los que están a menos de 0,05 del mejor fijo—:

gamma real estimado	0,0046	señal/ruido	0,38
oráculo entre contendientes	0,932		
mejor fijo	0,875		
premio máximo	+0,058		
piso de ruido (bootstrap)	+0,065		
premio NETO	-0,008		

Y **ninguna señal separa a los tres contendientes entre sí**: todas con $p > 0,29$. Sólo **la celda** lo logra ($p = 0,007$), y ésa no se conoce al decidir.

El 48% de interacción es real y vive **entre los brazos que nadie elegiría**. Reportar $\text{var}(\gamma)$ como evidencia de que rutear conviene mide la estructura equivocada.

7.11.4 Tampoco a nivel de familias

Una respuesta natural es que las señales quizá no separen individuos pero sí **grupos**: un ruteador que elige familia y después toma el más barato de la familia es otro ruteador, con otro premio. Se probó, declarando las familias **desde el código** —por cuándo el brazo decide su próxima llamada— y no desde el resultado:

familia	brazos	u media
adaptativo	react , reflection	0,818
plan_fijo	dag_strategy , rewoo	0,750
canal_con_pérdida	gist_reader , handoff , pointer_chase , supervisor	0,571

Valuar una familia por su **máximo** sería hacer trampa dos veces —le regala una elección por tarea que el ruteador de familias no puede hacer, y premia a la familia más numerosa por puro sesgo del máximo—, así que cada familia se representa por su brazo de mejor media, elegido **una vez** sobre todo el panel.

adaptativo	gana en 54 de 64 tareas	(84%)
plan_fijo	5 de 64	(8%)
canal_con_pérdida	5 de 64	(8%)
premio de rutear FAMILIAS	+0,079	(piso de ruido +0,065)
premio de rutear BRAZOS	+0,110	

Y **acá hay que hacer la resta que el paper acaba de enseñar**, porque los dos premios superan el piso: $+0,079 - 0,065 = +0,014$ para familias y $+0,110 - 0,065 = +0,045$ para brazos. Los dos netos son **positivos**, y aun así ninguno es cobrable — por una razón distinta de la de §7.11.3, y por eso hay que decirla:

allá el premio no existía; **acá existe y no hay con qué agarrarlo**. El premio es lo que capturaría un oráculo, y un oráculo no es una política: **ninguna señal predice la familia ganadora**. La mejor, cardinalidad, da información mutua 0,092 y **no sobrevive la corrección por selección** ($p = 0,365$).

Un premio sin señal que lo prediga es una cota superior, no un resultado. La distinción importa porque las dos secciones concluyen lo mismo por caminos opuestos, y presentarlas juntas sin la resta invita al lector a pensar que nos contradecemos.

No falla porque las señales sean débiles: falla porque no hay frontera que cruzar. Una familia que gana el 84% de las veces no es un cluster que rutear — es un default.

7.11.5 Lo que este corpus sí premia

Sobre calidad la decisión no tiene premio. Sobre **costo a utilidad igualada**, el mismo registro da un ahorro grande con una pérdida dentro del ruido, y a diferencia del premio de calidad **sobrevive evaluarlo fuera de muestra** (leave-one-task-out):

señal	utilidad vs mejor fijo	ahorro
cardinalidad × término	-0,017	42%
región	-0,110	74%
n_units	+0,000	1%

Es un intercambio, no una mejora: hay que decirlo así. Una medición previa nuestra sobre un panel más chico daba +0,008 de utilidad con 69% de ahorro —un almuerzo gratis— y **con el registro completo desapareció**. La pregunta «qué paradigma da la mejor respuesta» está agotada en este corpus; la pregunta «cuál es el más barato que da una respuesta indistinguible» no.

8. Mecanismos de falla

Éstos descansan en trazas y no en magnitudes, y son lo que defenderíamos.

8.1 La descomposición destruye la dependencia secuencial

Ambas topologías que descomponen fallan ambas celdas acopladas, y ambas **con la evidencia leída**. El DAG gastó 251.374 tokens en la cadena de tres saltos, leyó 4 de 4 unidades de la cadena, y sacó cero. Una cadena es secuencial por definición — el salto 2 no se puede formular antes de responder el salto 1 — así que partirla en sub-preguntas paralelas deja el blackboard con las unidades correctas y sin la estructura que las ordena. Es una propiedad de descomponer, no una coincidencia de dos implementaciones.

8.2 Los bucles iterativos amplifican la falla de recuperación en lugar de absorberla

En la celda de dos saltos el DAG emitió 31 búsquedas por palabra clave, leyó **cero** unidades relevantes, y gastó 142.694 tokens. Después de la tercera búsqueda el retriever ya había demostrado que no iba a encontrar el objetivo; el bucle de verificar-replanificar lo leyó como *buscar otra vez*.

Esto invierte una intuición de diseño. Los bucles de verificación se agregan **para** robustez; acá el bucle **es** el mecanismo de amplificación. Sin él la falla habría costado 10k tokens en lugar de 143k.

8.3 Una falla que ninguna herramienta arregla

Map-Reduce falla ambas celdas acopladas y lee cero unidades relevantes en las cuatro, porque por construcción ve cada unidad aislada. Una cadena y una comparación entre unidades son irresolubles así por muchas veces que se las examine.

El arreglo no es una herramienta mejor sino una negativa: el acoplamiento es una propiedad declarada de la tarea, así que la capa de factibilidad puede excluir al paradigma antes de gastar un token — lo que es más barato que cualquier cantidad de aprendizaje de que pierde.

Y el precio de esa falla escala con el corpus, la falla no. El mismo paradigma, sobre la misma celda, a dos tamaños de unidad:

corpus	unidades	llamadas	unid. relevantes leídas	costo	calidad
gold_v2	48	49	0	13.931	0,000
gold_deep	48	49	0	276.355	0,000

Cardinalidad idéntica, mismas llamadas, mismo cero — y **19,8× el costo**. Treinta veces más texto por unidad compró exactamente treinta veces más de nada, porque el obstáculo nunca fue la cantidad de evidencia. Es el argumento más filosófico que tenemos para decidir **antes** de correr y no después: un paradigma cuya limitación es estructural no falla más fuerte a escala, sólo más caro, y un enfoque que

aprende de resultados paga esa cuenta en cada episodio hasta aprender lo que la aritmética le podía decir gratis.

8.4 La atribución exige la traza

La traza de uso de herramientas separa *nunca vio la evidencia* de *la vio y razonó mal*. Sin ella un cero es un misterio; con ella, es un diagnóstico. "La topología que busca perdió" es ininterpretable cuando el buscador nunca llegó a una unidad relevante — eso es una afirmación sobre la superficie de herramientas, y sólo una falla **con la evidencia en la mano** es evidencia sobre la estructura de control.

9. Limitaciones y amenazas a la validez

El régimen en-ventana es casi tautológico, y §7.4 es la escapatoria — parcialmente recorrida. El corpus detrás de §7.1–7.3 tiene 16k tokens en su punto más ancho, así que leer-todo es correcto y lo más barato ahí. El régimen fuera-de-ventana ya está medido a 483k con `repeat = 3` (§7.4–7.5), y el claim de régimen a través de 135k/483k/1,27M descansa en el barrido de factibilidad a costo cero. Sigue abierto: no hay corrida completa a 1,27M (sólo su aritmética de factibilidad), dos celdas pesadas fuera de ventana todavía sin réplica completa (marcadas † en §7.4).

Y un punto salió de esa lista por haber sido contestado, en la dirección que nos cuesta. El mundo held-out con seed nueva —generado y verificado independientemente— ya corrió su test de transferencia registrado, P8. **Dos de sus cinco predicciones no transfieren**, así que por su propia regla de decisión registrada los veredictos por celda de más abajo son **corpus-locales**: afirmaciones sobre mundos seed-7, cada una con salvedad por mundo. La refutación no se apoya en las celdas que nadie resolvió — en las dos donde otro brazo llega a puntaje perfecto, el fallback general devuelve 0,667 y 0,000.

Ya existen réplicas, y el ruido es por celda. La réplica accidental del primer estudio (§7.3) y el `repeat = 3` del segundo coinciden: la reproducibilidad es por tarea — los lectores no dan vuelta nada, los que usan herramientas dan vuelta las celdas donde decide el retrieval, y el costo no fue reproducible en ninguna parte (dispersión media 2,05×, máxima 5,43× a calidad idéntica en el primer estudio; el mismo patrón de concentración en el segundo). Por eso todo delta de calidad se reporta con su conteo de flips por celda, nunca como media pelada. El piso de ruido formal por celda y la brecha de oráculo neta (`Study.noise_floor`, decisiones contra la brecha neta) se computan en el análisis final sobre la grilla completa — pendiente de las celdas en re-corrida.

Modelos: tres, y la perilla del sampleo resultó ser la preocupación equivocada. La primera grilla corrió sobre `gpt-5-chat`, que rechaza una temperatura explícita, así que el sampleo quedó en el default del modelo — reportado en su momento como la amenaza principal. Medido desde entonces sobre `gpt-5.4-nano`, `gpt-5.6-luna` y `gpt-5.6-terra`: los deployments de razonamiento rechazan `temperature` de plano, y `terra` en su propio default es el **más** reproducible de los tres. La restricción viva es otra y es estructural: en la familia `5.6`, `tools` y un `reasoning_effort` distinto de `none` no se pueden combinar en Chat Completions, y la diferencia que eso hace es total — 0,000 contra 1,000 sobre la misma tarea. Así que toda fila de campaña corre con `reasoning_effort = none`, que es un régimen declarado y no un default.

Corpus sintético. El ground truth es exacto y se re-deriva de forma independiente, y los parámetros estructurales son diales y no esperanzas — pero la distribución de tareas reales sobre esos diales es desconocida. Hacen falta benchmarks públicos para validez externa y no se usan todavía.

La latencia no es comparable. El DAG acá corre secuencial donde correría en oleadas concurrentes; y una vez que los workers compiten, el wall-clock por fila deja de medir latencia. La calidad y los conteos de tokens siguen siendo exactos.

La superficie de acciones es arquitectura declarada y no está ejercitada — dicho con el inventario, porque si no lo dice un lector lo encuentra. §5.5 gatea las acciones irreversibles con un piso de procedencia y §5.6 afirma la maquinaria sobre cuatro superficies. Una de esas cuatro nunca corrió. El

inventario completo de herramientas, sobre todos los corpus y los quince paradigmas registrados, son doce:

qué hacen	cuáles
leen el mundo	<code>search</code> <code>keyword_search</code> <code>semantic_search</code> <code>read</code> <code>read_all</code>
escriben el estado del propio agente	<code>note</code> <code>notes</code> <code>plan</code> <code>advance</code> <code>post</code> <code>board</code>
leen su propia contabilidad	<code>coverage</code>

Ninguna de las doce cambia nada fuera del proceso. No se escribe un archivo, no se manda un mensaje, no se actualiza una fila. Seis tareas de setenta y ocho llevan `irreversible = True` y tres llevan `shared_writes = True`, y esas banderas sí levantan el dial de garantía — pero la tarea que etiquetan es «*decidí si esta relación debe escalarse para congelamiento; contestá 'escalate' o 'no escalation'*», calificada por exact-match contra una clave. Es una clasificación sobre documentos con la etiqueta de una acción encima. No existe una herramienta que congele una cuenta, así que el piso que existe para gatear lo irreversible nunca tuvo un acto irreversible que gatear.

Lo enunciamos como amenaza en vez de resolverlo porque resolverlo es otro experimento, y porque la forma honesta de una afirmación ambiciosa es el balance que la acompaña: **\$5 se afirma para agentes en general y está verificado como tal; \$7–\$8 se afirman para extracción de respuesta exacta sobre documentos y se midieron ahí; la superficie de acciones está diseñada, tipada, probada en unidad y sin medir.** Lo más parecido a evidencia sobre una capacidad que no es recuperación es el resultado del blackboard en §7.7, y es un nulo.

Las afirmaciones de novedad están verificadas como conjunciones, no como partes. Los dos trabajos ancla se leyeron completos el 2026-08-26: todos los números citados de Select-then-Solve verifican contra su cuerpo, y la cesión a SCL se sostiene. Las afirmaciones restantes — la consolidación de la política de control (§6.3), el descubrimiento de proposiciones con compuerta de procedencia, la abstención en el ruteo de paradigmas — sobreviven búsquedas fechadas sólo como conjunciones cuyos conjuntos individuales tienen cada uno un vecino publicado (§2.2, §2.4, §2.6). Un campo que se mueve así de rápido puede cerrar cualquiera de ellas en meses; las búsquedas están fechadas para que un lector pueda recorrerlas.

Advertencia de reproducibilidad. Una semilla sola no fija un corpus. Cuando el algoritmo de generación cambió, la misma semilla produjo un mundo distinto; los manifiestos ahora estampan versión de generador y los resultados entre versiones no deben mezclarse.

10. Conclusión

El premio en la selección de paradigmas es real y está medido, y la literatura lo persigue con selectores que deben elegir siempre. Sostenemos que tres cosas van delante de ese problema. La factibilidad es aritmética y poda el espacio gratis, separando un límite de cardinalidad de un límite de contexto que se confunden de rutina. La selección paga sólo bajo una condición que podemos enunciar y verificar, y la condición implica abstenerse la mayor parte del tiempo. Y la varianza que se le atribuye a la elección de topología la gobierna en gran medida la superficie de herramientas: cincuenta veces entre los paradigmas que usan herramientas contra $1,2\times$ entre los que leen.

Nuestro resultado medido más fuerte no es sobre topologías en absoluto. Decirle a una topología iterativa que su retriever había dejado de producir cortó su costo $3,05\times$ contra una dispersión de réplica de $1,62\times$, con la caída localizada exactamente en las dos celdas desbocadas que el mecanismo predice. Una réplica no planeada además nos obligó a retirar un efecto reportado y partir otro por la mitad, que es la disciplina funcionando y no fallando. Las tres fallas de costo más grandes que encontramos no se arreglaron con topologías mejores ni con modelos mejores, sino con señales de contabilidad que no existían.

Lo que no tenemos es un selector validado, un piso de ruido, ni ninguna medición en el régimen donde leer todo es imposible. El harness para las tres cosas está construido y el protocolo está registrado de antemano, incluida la regla de decisión para el resultado en que ningún selector le gane al fallback — que sería un resultado, y se declara publicable **antes** de tener los datos.

Apéndice A — Artefactos

artefacto	contenido
el control nulo	el andamiaje por prompt se conserva en el registro y no se ejecuta nunca : dominado por <code>direct</code> en toda celda medida, a igual utilidad y jamás más barato. Es la evidencia de que andamiar por fraseo no compra nada, no un brazo
<code>PATTERNS.md</code>	catálogo de patrones: 10 estructurales, 4 de control, 15 anti-patrones, con aplicabilidad enunciada sobre el vector de features
<code>ANALYSIS.md</code>	el análisis de fallas de §8 completo, por paradigma y por celda
<code>GATE.md</code>	ocho criterios binarios de publicación y su veredicto actual
<code>PLAN.md</code>	historia de revisiones de la tesis, incluidos dos encuadres superados y por qué
<code>D:\Apps\MAPO\lab</code>	el harness: 15 paradigmas registrados , de los cuales 12 corren la campaña, 9 brazos de recuperación implementados de los cuales 2 corrieron alguna vez , 4 superficies de herramientas, 4 niveles de garantía, 12 herramientas, generador de corpus con verificador independiente, 573 aserciones chequeadas por máquina (521 + 52 en dos suites)

Siete de los quince anti-patrones del catálogo son errores cometidos y medidos en el curso de este trabajo, incluidos dos que contradijeron nuestras propias predicciones publicadas.